

上海健康医学院 医疗器械学院

学 生 实 验 报 告

课程名称： 健康大数据可视化 A

专业全称： 数据科学与大数据技术

班 级： 22 级数据科学与大数据技术 2 班

姓 名： 周雨晴

学 号： B22040302020

指导教师： 罗永刚

职 称： 讲师

实验日期： 2025. 11. 15



上海健康医学院
SHANGHAI UNIVERSITY OF MEDICINE & HEALTH SCIENCES

实验四 生物信息可视化

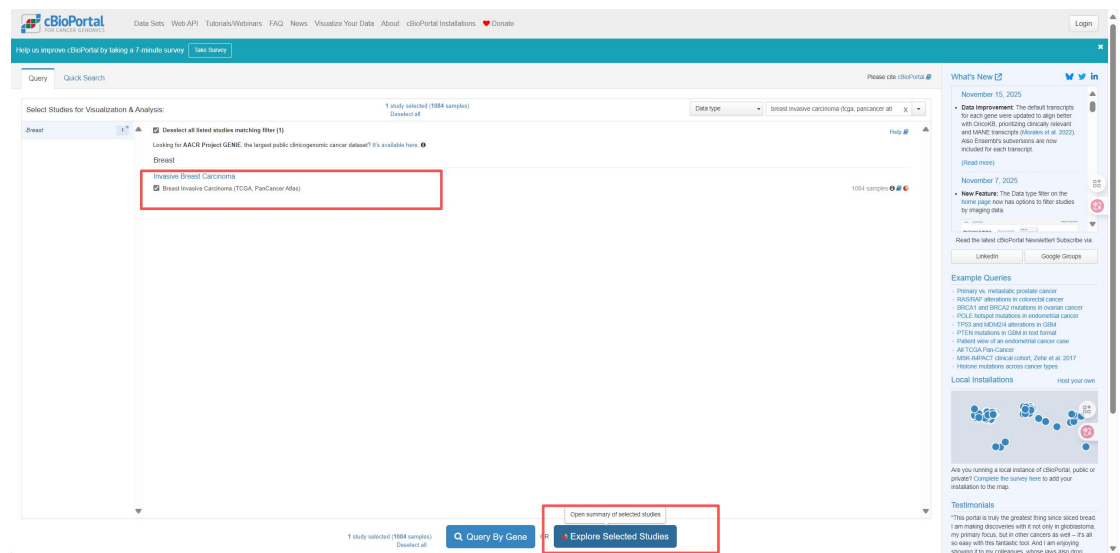
实验内容

完成实验指导书中的实验七、实验三中的时间相关部分，实验数据可以选择其它生物信息相关数据，在实验提报告中详细说明参考论文及其数据下载方法与途径。提交报告与数据代码。

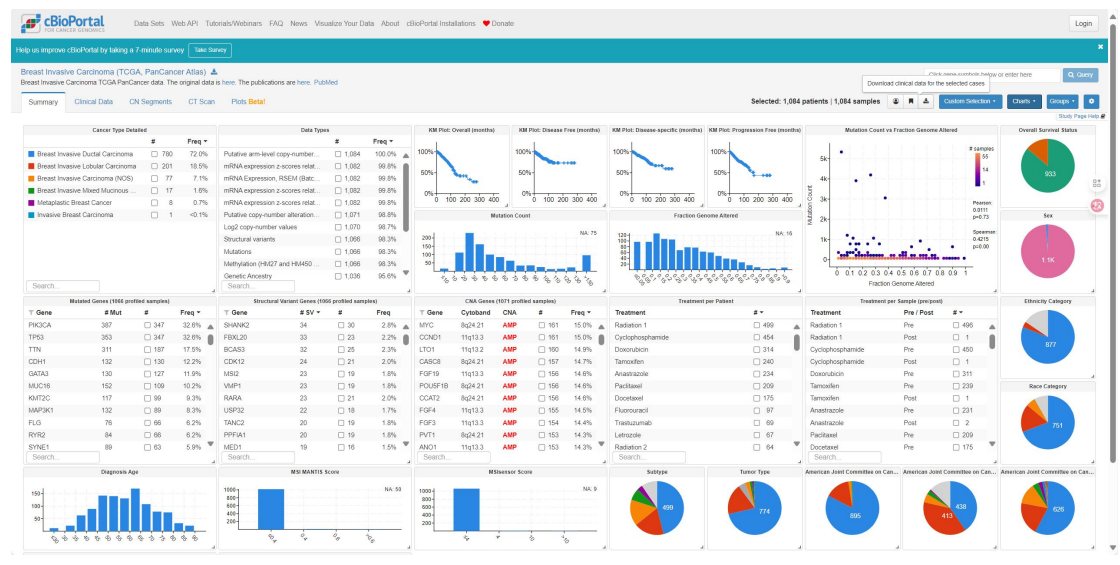
实验方法与步骤:

实验七 生物信息进行可视化

进入网站 <https://www.google.com/url?sa=E&q=https%3A%2F%2Fwww.cbioportal.org%2F> , 勾选数据集, 点击底部的 "Explore Selected Studies"



可以看到数据集的信息



下载数据集

Breast Invasive Carcinoma (TCGA, PanCancer Atlas) 

Breast Invasive Carcinoma TCGA PanCancer data. The original data is here. The processed data is here. Published

SummaryClinical DataON SegmentsCT ScanPlotsBetas

Selected: 1,064 patients | 1,064 samples

Custom SelectionChartsGenesState Page Help

brca_tcga_pan_can_atlas_2018

brca_tcga_pan_can_atlas_2018.tar.gzbrca_tcga_pan_can_atlas_2018

在 brca_tcga_pan_can_atlas_2018

新建

排序查看

预览

名称	类型	大小	修改日期
case_lists	文件夹		2024/12/27 4:16
normals	文件夹		2024/12/27 4:16
data_armlevel_cna.txt	文本文档	316 KB	2024/12/27 4:16
data_clinical_patient.txt	文本文档	334 KB	2024/12/27 4:16
data_clinical_sample.txt	文本文档	200 KB	2024/12/27 4:16
data_clinical_supp_hypoxia.txt	文本文档	25 KB	2024/12/27 4:16
data_cna.txt	文本文档	58,891 KB	2024/12/27 4:16
data_cna_hg19.seg	SEG 文件	9,938 KB	2024/12/27 4:16
data_gene_panel_matrix.txt	文本文档	30 KB	2024/12/27 4:16
data_genetic_ancestry.txt	文本文档	62 KB	2024/12/27 4:16
data_log2_cna.txt	文本文档	162,172 KB	2024/12/27 4:16
data_methylation_hm27_hm450_mer...	文本文档	434,776 KB	2024/12/27 4:16
data_mrna_seq_v2_rsem.txt	文本文档	151,030 KB	2024/12/27 4:16
data_mrna_seq_v2_rsem_zscores_ref_...	文本文档	160,688 KB	2024/12/27 4:16
data mrna seq v2 rsem zscores ref ...	文本文档	163,977 KB	2024/12/27 4:16

56 个项目

选择要预览的文件。

找到 data_clinical_patient.txt 和 data_clinical_sample.txt。

名称	修改日期	类型	大小
今天			
case_lists	2026/1/7 18:19	文件夹	
normals	2026/1/7 18:19	文件夹	
很久以前			
data_methylation_hm27_hm450_merged.txt	2024/12/27 4:16	文本文档	434,776 KB
data_armlevel_cna.txt	2024/12/27 4:16	文本文档	316 KB
data_clinical_patient.txt	2024/12/27 4:16	文本文档	334 KB
data_clinical_sample.txt	2024/12/27 4:16	文本文档	200 KB
data_clinical_supp_hypoxia.txt	2024/12/27 4:16	文本文档	25 KB
data_gene_panel_matrix.txt	2024/12/27 4:16	文本文档	30 KB
data_genetic_ancestry.txt	2024/12/27 4:16	文本文档	62 KB
data_mrna_seq_v2_rsem_zscores_ref_all_sa...	2024/12/27 4:16	文本文档	160,688 KB
data_resource_definition.txt	2024/12/27 4:16	文本文档	1 KB
data_resource_patient.txt	2024/12/27 4:16	文本文档	135 KB
data_timeline_sample_acquisition.txt	2024/12/27 4:16	文本文档	105 KB

关键列（Label）查找：

受体状态：查找 ER_STATUS, PR_STATUS, HER2_STATUS。

乳腺癌分期 数据导入

搜索

数据表格

Three groups

乳腺癌--分期

亚型

新的数据表格

信息

项目 信息1

新信息...

结果

生存 of Three groups

生存 of 乳腺癌--分期

生存 of 亚型

新的分析...

图

Three groups

乳腺癌--分期

亚型

新的图表

布局

新布局

系列

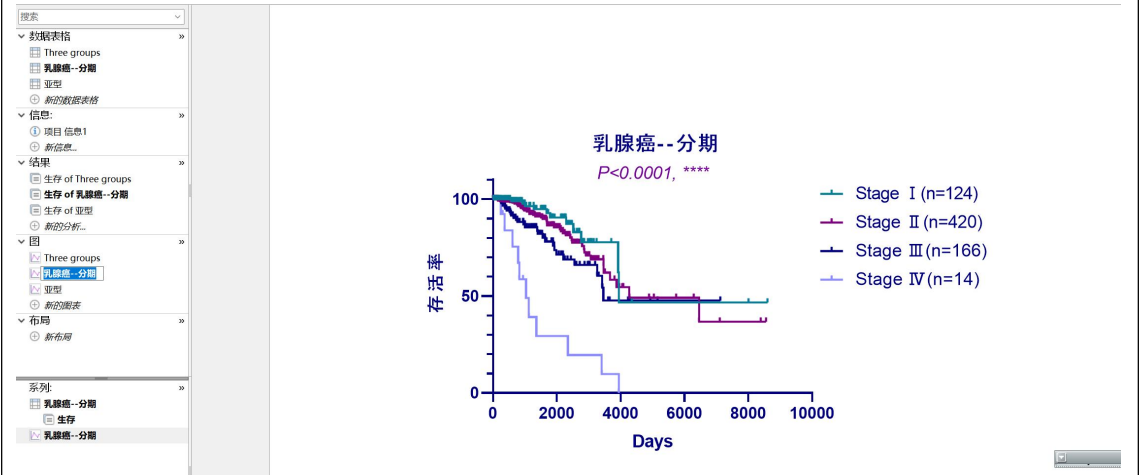
乳腺癌--分期

生存

乳腺癌--分期

格式: 生存		X	组 A	组 B	组 C	组 D	组 E	组 F	组 G
		Days	Stage I	Stage II	Stage III	Stage IV	标题	标题	标题
1	TCGA-E2-A106-01	2541	Y	Y	Y	Y	Y	Y	Y
2	TCGA-BH-A0HW-01	1561							
3	TCGA-A2-A0T3-01	1516							
4	TCGA-BH-A0BG-01	1871							
5	TCGA-BH-A0H6-01	747							
6	TCGA-D8-A1XA-01	839							
7	TCGA-AO-A0J4-01	1587							
8	TCGA-A2-A0EV-01	968							
9	TCGA-A7-A0CD-01	1165							
10	TCGA-E2-A15J-01	1640							
11	TCGA-AR-A1AK-01	3159							
12	TCGA-BH-A0B9-01	1572							
13	TCGA-B6-A0IP-01	3926	1						
14	TCGA-E2-A1IU-01	865							
15	TCGA-BH-A0HU-01	392							
16	TCGA-A2-A0SX-01	1534							
17	TCGA-B6-A0RU-01	8605							
18	TCGA-E2-A1IF-01	1138							
19	TCGA-E2-A15C-01	694							
20	TCGA-A7-A0DC-01	906							
21	TCGA-E2-A1LH-01	3247							
22	TCGA-B6-A1KI-01	2236							
23	TCGA-A1-A0SB-01	259							
24	TCGA-AR-A1AJ-01	3072							
25	TCGA-E2-A15O-01	1545							
26	TCGA-A1-A0SE-01	1321							
27	TCGA-E2-A156-01	726							
28	TCGA-AN-A0FS-01	210							
29	TCGA-BH-A0AV-01	1820							
30	TCGA-B6-A0X0-01	3945	1						
31	TCGA-BH-A0B8-01	1569							
32	TCGA-BH-A0C3-01	2709							

乳腺癌分期 生存分析可视化



乳腺癌亚型 数据导入

项目2.pzfx分期 - GraphPad 棱镜 8.0.2 (263)

文件 编辑 视图 插入 更改 排列 系列: 窗口 帮助

格式 文件 表格 数据 分析 更改 导入 绘图 写 文本 导出 打印 发送 LA 帮助

格式 文件 表格 数据 分析 更改 导入 绘图 写 文本 导出 打印 发送 LA 帮助

搜索

数据表格

信息

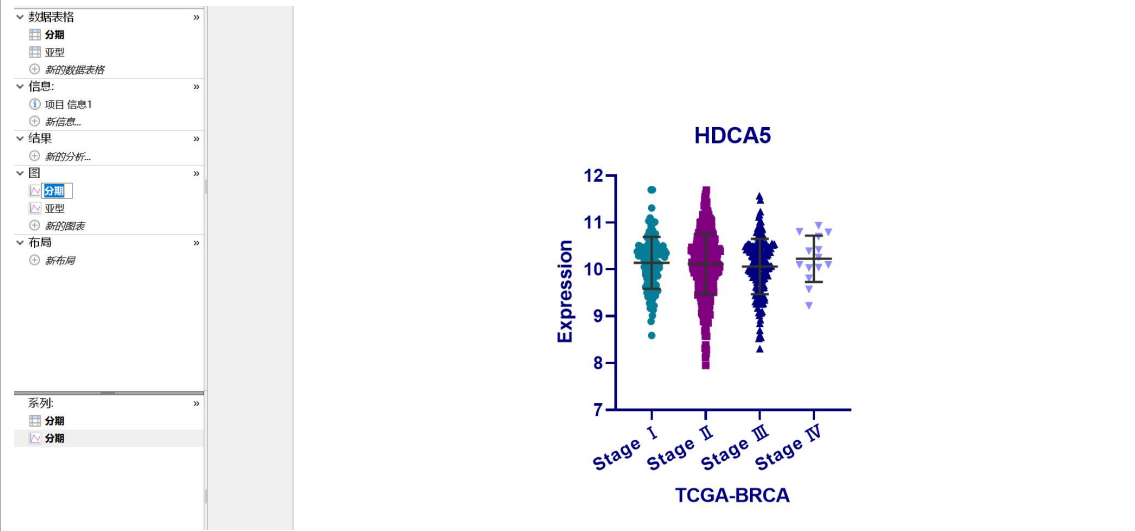
结果

图

布局

	组 A	组 B	组 C	组 D	组 E	组 F	组 G	组 H	组 I	组 J	组 K
	Stage I	Stage II	Stage III	Stage IV	标题	标题	标题	标题	标题	标题	标题
1	11.700	11.690	11.570	10.930							
2	11.700	11.610	11.480	10.800							
3	11.310	11.550	11.230	10.790							
4	11.100	11.500	11.170	10.700							
5	11.040	11.490	11.120	10.420							
6	11.030	11.440	11.020	10.390							
7	11.010	11.400	11.020	10.250							
8	10.990	11.390	10.970	10.100							
9	10.920	11.380	10.920	10.100							
10	10.840	11.360	10.880	10.040							
11	10.810	11.270	10.880	10.030							
12	10.760	11.260	10.880	9.809							
13	10.760	11.250	10.850	9.572							
14	10.730	11.230	10.830	9.221							
15	10.730	11.220	10.800								
16	10.710	11.210	10.790								
17	10.680	11.170	10.740								
18	10.660	11.160	10.720								
19	10.660	11.160	10.710								
20	10.590	11.150	10.700								
21	10.560	11.140	10.660								
22	10.550	11.130	10.620								
23	10.550	11.120	10.610								

乳腺癌分期 四阶段数据可视化



乳腺癌亚型 四阶段数据导入

项目2.pzfx-亚型 - GraphPad 棱镜 8.0.2 (263)

文件 编辑 视图 插入 更改 排列 系列: 窗口 帮助

快捷 文件 表格 数据 排列 分析 更改 导入 绘图 写 文本 导出 打印 发送 LA 帮助

搜索

数据表格

分期

亚型

新的数据表格

信息:

项目 信息1

新信息...

结果

新的分析...

图

分期

亚型

新的图表

布局

新布局

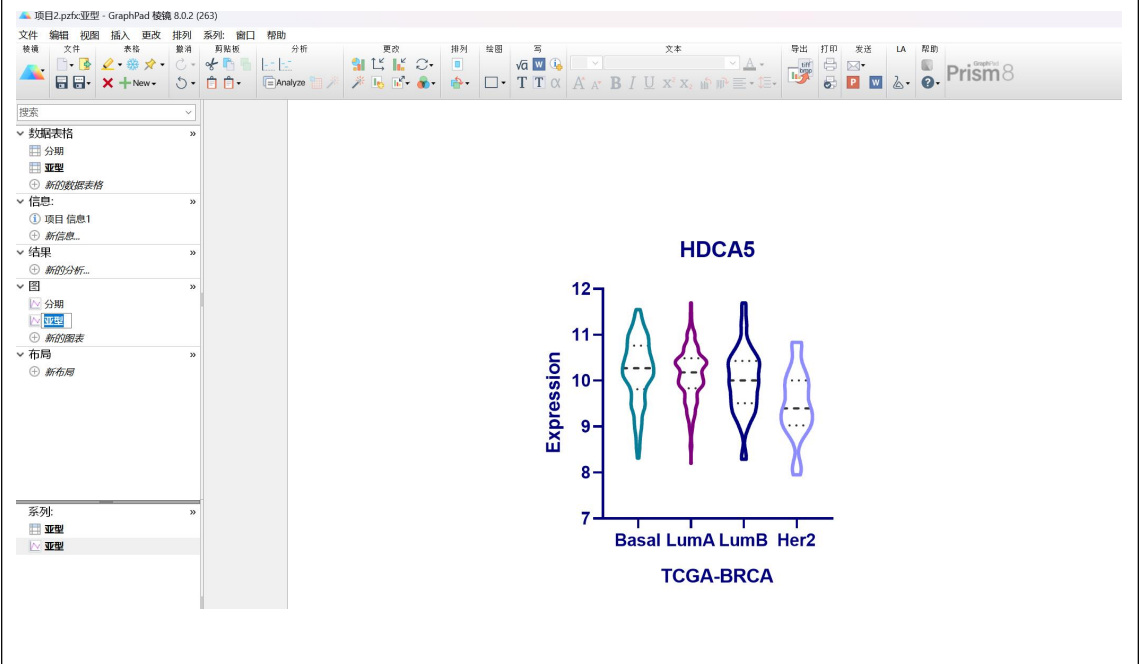
系列:

亚型

亚型

	组 A	组 B	组 C	组 D	组 E	组 F	组 G	组 H	组 I	组 J	组 K
	Basal	LumA	LumB	Her2	标题	标题	标题	标题	标题	标题	标题
1	11.550	11.700	11.700	10.840							
2	11.500	11.690	11.610	10.830							
3	11.400	11.570	11.480	10.700							
4	11.390	11.490	11.230	10.410							
5	11.360	11.440	11.230	10.240							
6	11.270	11.380	11.160	10.200							
7	11.220	11.310	10.880	10.130							
8	11.210	11.260	10.840	10.030							
9	11.170	11.250	10.800	10.000							
10	11.140	11.170	10.740	9.920							
11	11.100	11.160	10.7	9.899							
12	11.090	11.150	10.660	9.797							
13	11.060	11.130	10.620	9.781							
14	11.020	11.120	10.590	9.732							
15	11.020	11.120	10.550	9.587							
16	11.010	11.090	10.520	9.546							
17	10.990	11.060	10.510	9.451							
18	10.990	11.050	10.510	9.346							
19	10.970	11.050	10.500	9.345							
20	10.930	11.040	10.490	9.327							
21	10.920	11.040	10.470	9.316							
22	10.920	11.030	10.420	9.277							
23	10.880	11.020	10.420	9.216							
24	10.880	11.010	10.380	9.082							
25	10.860	11.010	10.370	9.063							
26	10.830	11.010	10.370	9.030							
27	10.810	11.010	10.330	9.025							
28	10.800	11.000	10.320	9.023							

乳腺癌亚型 四阶段数据可视化



实验三：基于 Covid-19 的时变数据可视化

单城市分析：

运行代码：

```
1. import pandas as pd
2. import numpy as np
3. import matplotlib.pyplot as plt
4. import seaborn as sns
5. import os
6. import matplotlib.dates as mdates
7. import matplotlib.ticker as ticker
8.
9.
10. # =====
11. # 1. 风格配置
12. # =====
13. def set_pub_style():
14.     sns.set_context("paper", font_scale=1.2)
15.     sns.set_style("ticks")
16.     plt.rcParams['font.family'] = 'sans-serif'
17.     plt.rcParams['font.sans-serif'] = ['Arial', 'SimHei', 'Microsoft YaHei', 'DejaVu Sans']
18.     plt.rcParams['axes.unicode_minus'] = False
19.
20.
21. set_pub_style()
22.
23.
24. # =====
25. # 2. 数据加载模块
26. # =====
27. class CityDataLoader:
28.     def __init__(self, base_path, city_name="city_A"):
29.         self.city_path = os.path.join(base_path, city_name)
30.         self.city_name = city_name
31.
32.     def normalize_to_relative_day(self, df, col_name='date'):
33.         """将不同格式的日期（YYYYMMDD 或 0,1,2）统一转换为 relative_day"""
34.         if df.empty or col_name not in df.columns:
35.             return df
```

```

36.
37.         try:
38.             # 先强制转为数值，无法转换的变 NaN
39.             numeric_dates = pd.to_numeric(df[col_name], errors='coerce')
40.
41.             # 判断是否解析成功
42.             if numeric_dates.isna().all():
43.                 # 如果全是 NaN，说明可能是字符串日期 "2020-01-01"
44.                 temp_dt = pd.to_datetime(df[col_name], errors='coerce')
45.                 min_dt = temp_dt.min()
46.                 if pd.isna(min_dt):
47.                     # 极端的兜底：如果完全无法解析，就生成序列索引
48.                     df['relative_day'] = range(len(df))
49.                     df['date_dt'] = pd.to_datetime('2020-01-01') + pd.to_timedelta(df['relative_day'], unit='D')
50.                 else:
51.                     df['relative_day'] = (temp_dt - min_dt).dt.days
52.                     df['date_dt'] = temp_dt
53.
54.             elif numeric_dates.max() > 10000:
55.                 # YYYYMMDD 格式 (如 20200101)
56.                 temp_dt = pd.to_datetime(numeric_dates, format='%Y%m%d', errors='coerce')
57.                 min_dt = temp_dt.min()
58.                 df['relative_day'] = (temp_dt - min_dt).dt.days
59.                 df['date_dt'] = temp_dt
60.
61.             else:
62.                 # 已经是相对天数 (0, 1, 2)
63.                 df['relative_day'] = numeric_dates
64.                 # 构造伪日期用于 X 轴显示
65.                 df['date_dt'] = pd.to_datetime('2020-01-01') + pd.to_timedelta(numeric_dates, unit='D')
66.
67.         except Exception as e:
68.             print(f"Date normalization warning: {e}")
69.             df['relative_day'] = 0
70.
71.         return df
72.
73.     def load_all(self):
74.         print(f"Loading data for {self.city_name} from {self.city_path}...")
75.
76.         # 1. Infection

```

```

77.         f_inf = os.path.join(self.city_path, 'infection.csv')
78.         if os.path.exists(f_inf):
79.             df_inf = pd.read_csv(f_inf, names=['date', 'region_id', 'new_infections'], header=None)
80.             df_inf = self.normalize_to_relative_day(df_inf, 'date')
81.         else:
82.             print(f"Error: {f_inf} not found.")
83.             df_inf = pd.DataFrame()
84.
85.         # 2. Migration
86.         f_mig = os.path.join(self.city_path, 'migration.csv')
87.         if os.path.exists(f_mig):
88.             df_mig = pd.read_csv(f_mig, names=['date', 'from_city', 'to_city', 'mig_index'], header=None)
89.             df_mig = self.normalize_to_relative_day(df_mig, 'date')
90.         else:
91.             df_mig = pd.DataFrame()
92.
93.         # 3. Density (限制行数)
94.         f_den = os.path.join(self.city_path, 'density.csv')
95.         if os.path.exists(f_den):
96.             df_den = pd.read_csv(f_den, names=['date', 'hour', 'lon', 'lat', 'f_low_index'], header=None, nrows=200000)
97.             df_den = self.normalize_to_relative_day(df_den, 'date')
98.         else:
99.             df_den = pd.DataFrame()
100.
101.         # 4. Grid Attribute
102.         f_grid = os.path.join(self.city_path, 'grid_attr.csv')
103.         if os.path.exists(f_grid):
104.             df_grid = pd.read_csv(f_grid, names=['lon', 'lat', 'region_id'], header=None)
105.         else:
106.             df_grid = pd.DataFrame()
107.
108.         # 5. Weather
109.         f_wea = os.path.join(self.city_path, 'weather.csv')
110.         if os.path.exists(f_wea):
111.             df_wea = pd.read_csv(f_wea, names=['date', 'hour', 'temp', 'humidity', 'wind_dir', 'wind_spd', 'wind_force',
112.                                                'weather_type'], header=None)
113.             df_wea = self.normalize_to_relative_day(df_wea, 'date')
114.         else:

```

```

115.         df_wea = pd.DataFrame()
116.
117.         print("Data loaded successfully.")
118.         return df_inf, df_mig, df_den, df_grid, df_wea
119.
120.
121. # =====
122. # 3. 可视化绘图模块
123. # =====
124.
125. def plot_epidemic_migration_dynamics(df_inf, df_mig, city_name):
126.     """ 图 1: 疫情与迁徙 """
127.     if 'relative_day' not in df_inf.columns or 'relative_day' not in df_mig.
        columns:
128.         return
129.
130.     daily_inf = df_inf.groupby('relative_day')['new_infections'].sum().reset
        _index()
131.     daily_mig = df_mig.groupby('relative_day')['mig_index'].sum().reset_inde
        x()
132.
133.     # 映射日期
134.     if 'date_dt' in df_inf.columns:
135.         date_map = df_inf[['relative_day', 'date_dt']].drop_duplicates().set
            _index('relative_day')
136.     else:
137.         # Fallback
138.         daily_inf['date_dt'] = daily_inf['relative_day']
139.         date_map = None
140.
141.     merged = pd.merge(daily_inf, daily_mig, on='relative_day', how='inner')
142.
143.     if date_map is not None:
144.         merged['date_dt'] = merged['relative_day'].map(date_map['date_dt'])
145.
146.     if merged.empty:
147.         print("Plot 1 Skipped: No overlapping dates.")
148.         return
149.
150.     fig, ax1 = plt.subplots(figsize=(10, 5))
151.
152.     color_inf = '#E63946'

```

```

153.     ax1.bar(merged['date_dt'], merged['new_infections'], color=color_inf, al
        pha=0.6, label='Infections')
154.     ax1.set_ylabel('New Infections', color=color_inf, fontweight='bold')
155.     ax1.tick_params(axis='y', labelcolor=color_inf)
156.
157.     # 日期格式化
158.     if date_map is not None:
159.         ax1.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d'))
160.         ax1.xaxis.set_major_locator(mdates.DayLocator(interval=5))
161.
162.     ax2 = ax1.twinx()
163.     color_mig = '#1D3557'
164.     ax2.plot(merged['date_dt'], merged['mig_index'], color=color_mig, marker
        ='o', markersize=4, label='Migration')
165.     ax2.set_ylabel('Migration Index', color=color_mig, fontweight='bold')
166.     ax2.tick_params(axis='y', labelcolor=color_mig)
167.
168.     plt.title(f'Dynamics in {city_name}')
169.     plt.tight_layout()
170.     plt.show()
171.
172.
173. def plot_spatial_heatmap(df_inf):
174.     """ 图 2: 时空热力图 """
175.     print("Preparing Spatial Heatmap...")
176.     if df_inf.empty: return
177.
178.     pivot = df_inf.pivot_table(index='region_id', columns='date_dt', values=
        'new_infections', aggfunc='sum').fillna(0)
179.
180.     if pivot.size == 0:
181.         print("Error: Pivot table is empty.")
182.         return
183.
184.     plt.figure(figsize=(12, 6))
185.
186.     x_labels = [d.strftime('%m-%d') if hasattr(d, 'strftime') else str(d) fo
        r d in pivot.columns]
187.
188.     ax = sns.heatmap(pivot, cmap='YlOrRd', linewidths=0.0, xticklabels=x_lab
        els,
189.                      cbar_kws={'label': 'New Infections'})
190.
191.     plt.title('Spatiotemporal Distribution of Infections')

```

```

192.     plt.ylabel('Region ID')
193.     plt.xlabel('Date')
194.
195.     if len(pivot.columns) > 20:
196.         ticks = ax.get_xticks()
197.         labels = ax.get_xticklabels()
198.         n = 5
199.         ax.set_xticks(ticks[::n])
200.         ax.set_xticklabels(labels[::n], rotation=0)
201.
202.     plt.tight_layout()
203.     plt.show()
204.
205.
206. def plot_grid_density_scatter(df_grid, df_den):
207.     """ 图 3: 网格分布 """
208.     print("Preparing Grid Map...")
209.     if df_grid.empty or df_den.empty: return
210.
211.     # 强制转为数值, 防止 merge 失败
212.     for df in [df_grid, df_den]:
213.         df['lon'] = pd.to_numeric(df['lon'], errors='coerce')
214.         df['lat'] = pd.to_numeric(df['lat'], errors='coerce')
215.
216.     grid_flow = df_den.groupby(['lon', 'lat'])['flow_index'].mean().reset_in
dex()
217.
218.     df_grid['lon_r'] = df_grid['lon'].round(4)
219.     df_grid['lat_r'] = df_grid['lat'].round(4)
220.     grid_flow['lon_r'] = grid_flow['lon'].round(4)
221.     grid_flow['lat_r'] = grid_flow['lat'].round(4)
222.
223.     merged = pd.merge(df_grid, grid_flow, on=['lon_r', 'lat_r'], how='inner'
)
224.
225.     if merged.empty:
226.         merged = df_grid
227.         size_col = None
228.     else:
229.         size_col = 'flow_index'
230.
231.     plt.figure(figsize=(8, 7))
232.     sns.scatterplot(data=merged, x='lon_x', y='lat_x', hue='region_id',
233.                     size=size_col, palette='tab20', legend=False, alpha=0.8,

```



```

    edgecolor=None)
234.     plt.title('Spatial Grid Distribution')
235.     plt.axis('equal')
236.     plt.show()
237.
238.
239. def plot_weather_correlation(df_inf, df_wea):
240.     """ 图 4: 天气相关性 (已彻底修复类型错误) """
241.     print("Preparing Weather Correlation...")
242.     if df_inf.empty or df_wea.empty:
243.         print("Weather or Infection data missing.")
244.         return
245.
246.     target_cols = ['temp', 'humidity', 'wind_spd', 'wind_force']
247.     available_cols = [c for c in target_cols if c in df_wea.columns]
248.
249.     if not available_cols:
250.         print("No weather columns found.")
251.         return
252.
253.     # =====
254.     # 核心修复: 强制将天气列转换为数值类型
255.     # =====
256.     for col in available_cols:
257.         # errors='coerce' 会将所有非数字字符变成 NaN, 从而避免 groupby 报错
258.         df_wea[col] = pd.to_numeric(df_wea[col], errors='coerce')
259.
260.     # 筛选出还有有效数据的列
261.     valid_cols = [c for c in available_cols if df_wea[c].notna().any()]
262.
263.     if not valid_cols:
264.         print("Error: Weather columns contain no numeric data.")
265.         return
266.
267.     # 现在列里全是数字, mean() 绝对安全
268.     daily_wea = df_wea.groupby('relative_day')[valid_cols].mean().reset_index()
269.
270.     daily_inf = df_inf.groupby('relative_day')['new_infections'].sum().reset_index()
271.
272.     merged = pd.merge(daily_inf, daily_wea, on='relative_day')
273.
274.     if merged.shape[0] < 2:

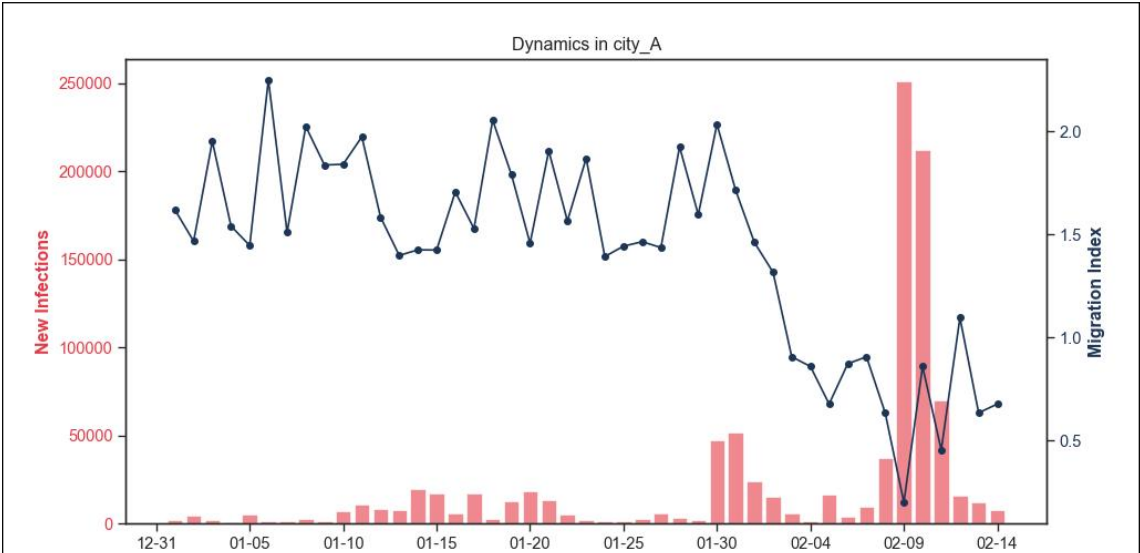
```

```

275.         print("Not enough data points for correlation.")
276.         return
277.
278.     corr = merged.drop(columns=['relative_day']).corr()
279.
280.     plt.figure(figsize=(8, 6))
281.     sns.heatmap(corr, cmap='coolwarm', annot=True, fmt=".2f", center=0,
282.                 linewidths=0.5, cbar_kws={"shrink": .7})
283.     plt.title('Correlation: Weather Factors vs. Infection')
284.     plt.tight_layout()
285.     plt.show()
286.
287.
288. # =====
289. # 4. 主程序
290. # =====
291. if __name__ == "__main__":
292.     BASE_DIR = r"C:\Users\21626\Downloads\train_data\train_data"
293.     TARGET_CITY = "city_A"
294.
295.     if not os.path.exists(os.path.join(BASE_DIR, TARGET_CITY)):
296.         print(f"Path Error: {os.path.join(BASE_DIR, TARGET_CITY)} does not exist.")
297.     else:
298.         try:
299.             loader = CityDataLoader(BASE_DIR, TARGET_CITY)
300.             df_inf, df_mig, df_den, df_grid, df_wea = loader.load_all()
301.
302.             plot_epidemic_migration_dynamics(df_inf, df_mig, TARGET_CITY)
303.             plot_spatial_heatmap(df_inf)
304.             plot_grid_density_scatter(df_grid, df_den)
305.             plot_weather_correlation(df_inf, df_wea)
306.
307.             print("All visualizations complete.")
308.         except Exception as e:
309.             print(f"An error occurred: {e}")
310.             import traceback
311.
312.             traceback.print_exc()

```

绘图结果:



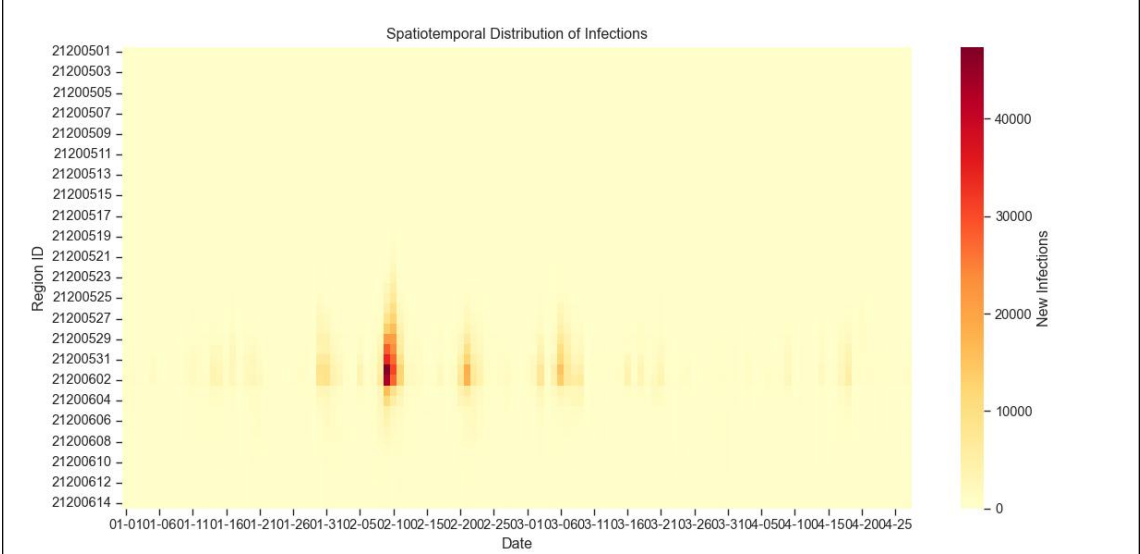
plot_epidemic_migration_dynamics（疫情与迁徙动态双轴图）

核心作用

- 时间维度对比：以“每日新增感染数（柱状图）+ 迁徙指数（折线图）”的双 Y 轴形式，展示 45 天内目标城市疫情传播与人口迁徙的时序变化；
- 量纲适配：解决“感染人数（绝对数）”与“迁徙指数（相对值）”量纲不同的可视化难题；
- 时间标注：格式化日期轴（月 - 日），聚焦核心时间节点的趋势关联。

意义

- 直观验证“人口迁徙是否为疫情传播的驱动因素”：可观察迁徙指数高峰是否滞后 / 同步于感染高峰（如迁徙指数上升后 7 天感染数激增）；
- 识别关键时间节点：如疫情爆发拐点、迁徙管制政策生效时间点，为因果分析提供可视化证据；
- 单城市基线分析：为后续跨城市对比（city_A-E）奠定基础。



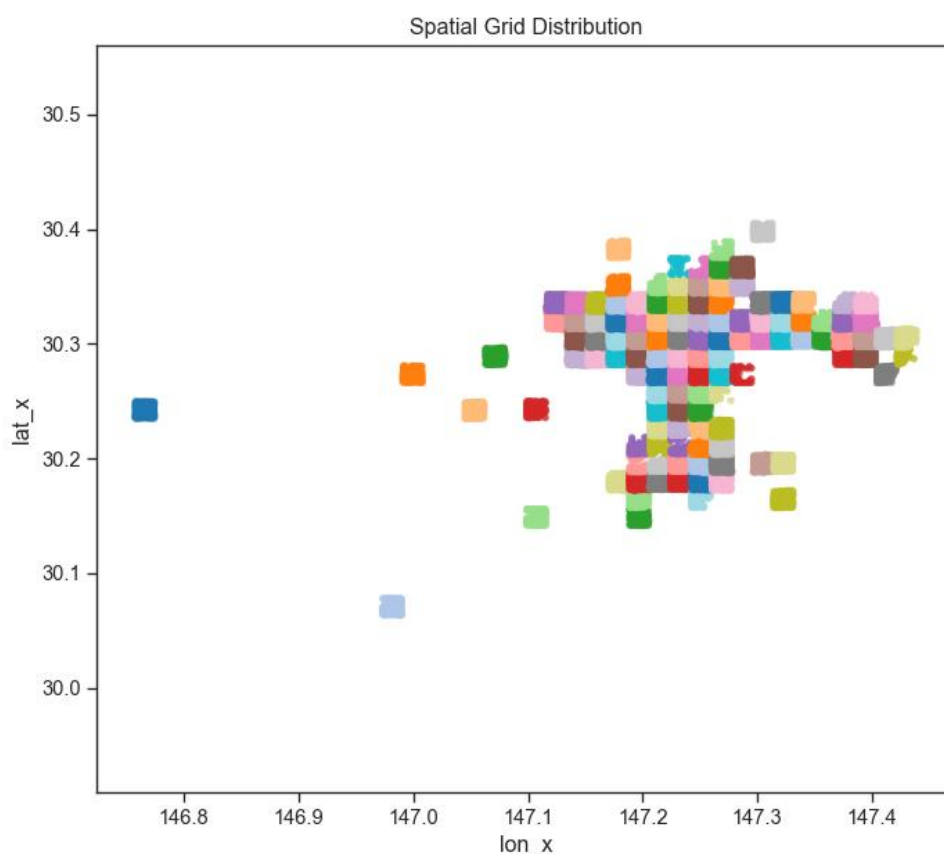
plot_spatial_heatmap（感染时空热力图）

核心作用

- 时空融合：以“区域 ID（纵轴）× 日期（横轴）”的热力图展示新增感染数分布，颜色深浅量化感染强度；
- 可读性优化：对密集的时间轴抽样（每 5 天标注），避免视觉拥挤。

意义

- 空间扩散规律：识别疫情的核心爆发区域、扩散方向（如从市中心区域向郊区蔓延）；
- 时空聚类：发现“高感染区域 - 时间”聚类（如某区域连续 7 天感染数居高），为区域化防控提供依据；
- 弥补单维度缺陷：结合时间 + 空间，解决纯时间（仅看总量）或纯空间（仅看静态分布）分析的片面性。



plot_grid_density_scatter（网格分布散点图）

核心作用

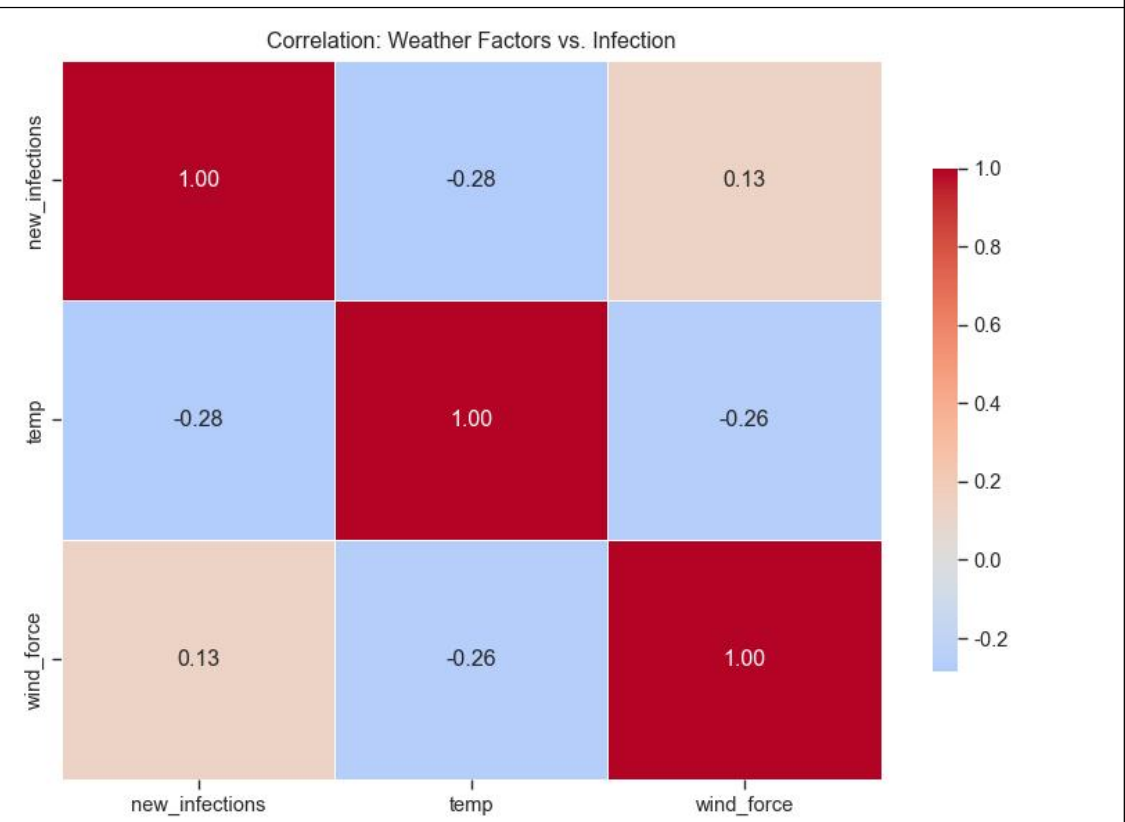
- 地理关联：结合网格经纬度、区域 ID、人流量指数，以散点图展示网格的空间分布；
- 维度映射：散点颜色对应区域 ID、大小对应人流量指数，直观呈现“位置 - 归属 - 密度”的关联。

意义

- 人口密度空间格局：识别城市人流量高值区（如商圈、交通枢纽），为疫情传播的“空间

易感区”分析提供基础；

- 网格 - 区域关联：验证网格归属的合理性，以及区域内人流量的空间异质性；
- 基础地理可视化：为后续“人流量 - 感染数”空间关联分析铺垫。



plot_weather_correlation（天气 - 感染相关性热力图）

核心作用

- 量化关联：计算气温、湿度、风速、风力与新增感染数的相关系数，以热力图展示相关性矩阵；
- 鲁棒性处理：强制将天气列转为数值类型，避免非数值数据导致的报错。

意义

- 环境因素筛选：识别与疫情传播显著相关的气象因素（如高湿度 / 低风速是否促进病毒传播）；
- 特征工程依据：为预测模型筛选关键气象特征（如仅保留显著相关的湿度、风速）；
- 机制分析：为“环境 - 病毒存活 - 疫情传播”的理论分析提供量化证据。

运行代码：

```
1. import pandas as pd
2. import numpy as np
3. import matplotlib.pyplot as plt
4. import seaborn as sns
5. import os
```

```
6. import matplotlib.dates as mdates
7. # 修正导入: 是 MinMaxScaler
8. from sklearn.preprocessing import MinMaxScaler
9.
10. def set_pub_style():
11.     sns.set_context("paper", font_scale=1.3)
12.     sns.set_style("whitegrid", {'grid.linestyle': '--'})
13.     plt.rcParams['font.family'] = 'sans-serif'
14.     # 兼容多种中文字体, 防止乱码
15.     plt.rcParams['font.sans-serif'] = ['Arial', 'SimHei', 'Microsoft YaHei', 'DejaVu Sans']
16.     plt.rcParams['axes.unicode_minus'] = False
17.
18.
19. set_pub_style()
20.
21.
22. # =====
23. # 1. 数据加载模块
24. # =====
25. class CityDataLoader:
26.     def __init__(self, base_path, city_name="city_A"):
27.         self.city_path = os.path.join(base_path, city_name)
28.
29.     def normalize_date(self, df, col='date'):
30.         """将日期标准化为相对天数和伪日期对象"""
31.         if df.empty or col not in df.columns: return df
32.
33.         # 强制转数值 (处理 '20200101' 或 '0, 1, 2')
34.         # errors='coerce' 会把无法转换的变成 NaN, fillna(0) 填充
35.         df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)
36.
37.         # 如果数值很大(如 20200101), 说明是绝对日期, 需要转换成相对天数
38.         if df[col].max() > 10000:
39.             temp = pd.to_datetime(df[col], format='%Y%m%d', errors='coerce')
40.             min_date = temp.min()
41.             # 覆盖原列为相对天数 (int)
42.             df[col] = (temp - min_date).dt.days.fillna(0).astype(int)
43.         else:
44.             # 已经是相对天数
45.             df[col] = df[col].astype(int)
46.
47.         # 构造一个伪时间用于绘图 (Base: 2020-01-01)
48.         df['dt'] = pd.to_datetime('2020-01-01') + pd.to_timedelta(df[col], unit
```

```

        ='D')
49.         return df
50.
51.     def load_temporal_data(self):
52.         print("Loading data...")
53.
54.         # 1. Infection (Daily)
55.         f_inf = os.path.join(self.city_path, 'infection.csv')
56.         if os.path.exists(f_inf):
57.             df_inf = pd.read_csv(f_inf, names=['date', 'region', 'cnt'], header
=None)
58.             df_inf = self.normalize_date(df_inf)
59.         else:
60.             df_inf = pd.DataFrame()
61.
62.         # 2. Migration (Daily)
63.         f_mig = os.path.join(self.city_path, 'migration.csv')
64.         if os.path.exists(f_mig):
65.             df_mig = pd.read_csv(f_mig, names=['date', 'f', 't', 'idx'], header
=None)
66.             df_mig = self.normalize_date(df_mig)
67.         else:
68.             df_mig = pd.DataFrame()
69.
70.         # 3. Density (Hourly) - 限制行数防止内存溢出
71.         f_den = os.path.join(self.city_path, 'density.csv')
72.         if os.path.exists(f_den):
73.             df_den = pd.read_csv(f_den, names=['date', 'hour', 'lon', 'lat', 'i
dx'], header=None, nrows=500000)
74.             df_den = self.normalize_date(df_den)
75.         else:
76.             df_den = pd.DataFrame()
77.
78.         return df_inf, df_mig, df_den
79.
80.
81. # =====
82. # 2. 高级时序可视化模块
83. # =====
84.
85. def plot_hourly_pulse(df_den):
86.     """图 1: 城市脉搏 (小时级变化)"""
87.     print("Plotting Hourly Pulse...")
88.     if df_den.empty:

```



```

89.         print("Skipping: Density data is empty.")
90.         return
91.
92.     # 聚合：计算每一天、每一小时的平均人流密度
93.     hourly_trend = df_den.groupby(['date', 'hour'])['idx'].mean().reset_index()
94.
95.     plt.figure(figsize=(10, 6))
96.
97.     # 绘制带置信区间的折线图
98.     sns.lineplot(data=hourly_trend, x='hour', y='idx',
99.                  color='#2A9D8F', linewidth=2.5, marker='o')
100.
101.     plt.title('Intraday Mobility Pattern: The "Pulse" of the City', fontsize
102.              =14, pad=15)
103.     plt.xlabel('Hour of Day (0-23)', fontweight='bold')
104.     plt.ylabel('Average Flow Index', fontweight='bold')
105.     plt.xticks(range(0, 24, 2))
106.
107.     # 标记白天黑夜
108.     plt.axvspan(0, 6, color='gray', alpha=0.1, label='Night')
109.     plt.axvspan(18, 23, color='gray', alpha=0.1)
110.
111.     plt.tight_layout()
112.     plt.show()
113.
114. def plot_normalized_evolution(df_inf, df_mig, df_den):
115.     """图 2：归一化协同演变"""
116.     print("Plotting Normalized Evolution...")
117.     if df_inf.empty or df_mig.empty:
118.         print("Skipping: Inf or Mig data is empty.")
119.         return
120.
121.     # 聚合每日数据
122.     d_inf = df_inf.groupby('date')['cnt'].sum().reset_index().rename(columns
123.                               ={'cnt': 'Infection'})
124.
125.     d_mig = df_mig.groupby('date')['idx'].sum().reset_index().rename(columns
126.                               ={'idx': 'Migration'})
127.
128.     if not df_den.empty:
129.         d_den = df_den.groupby('date')['idx'].mean().reset_index().rename(co
130.                                   lumns={'idx': 'Density'})
131.     else:

```

```

128.         d_den = pd.DataFrame(columns=['date', 'Density'])
129.
130.         # 合并所有数据
131.         merged = pd.merge(d_inf, d_mig, on='date', how='outer')
132.         if not d_den.empty:
133.             merged = pd.merge(merged, d_den, on='date', how='outer')
134.
135.         merged = merged.sort_values('date').fillna(0)
136.
137.         # 归一化处理 (Min-Max Scaling)
138.         scaler = MinMaxScaler()
139.         cols_to_norm = [c for c in ['Infection', 'Migration', 'Density'] if c in
            merged.columns]
140.
141.         # 防止全 0 列导致除以 0 警告
142.         if not merged.empty:
143.             merged[cols_to_norm] = scaler.fit_transform(merged[cols_to_norm])
144.
145.         # 转换为长格式以便绘图
146.         melted = merged.melt(id_vars='date', value_vars=cols_to_norm, var_name='
            Type', value_name='Normalized Value')
147.
148.         plt.figure(figsize=(12, 6))
149.
150.         palette = {'Infection': '#E63946', 'Migration': '#457B9D', 'Density': '#
            2A9D8F'}
151.
152.         sns.lineplot(data=melted, x='date', y='Normalized Value', hue='Type',
153.             palette=palette, linewidth=2.5, style='Type', markers=True,
            dashes=False)
154.
155.         plt.title('Co-evolution of Epidemic and Mobility (Normalized [0-1])', fo
            ntsize=14)
156.         plt.xlabel('Days (Relative)', fontweight='bold')
157.         plt.ylabel('Normalized Intensity', fontweight='bold')
158.
159.         # 标注最高点
160.         try:
161.             peak_inf = merged.loc[merged['Infection'].idxmax()]
162.             plt.annotate(f"Infection Peak (Day {int(peak_inf['date'])})",
163.                 xy=(peak_inf['date'], peak_inf['Infection']),
164.                 xytext=(peak_inf['date'] + 2, peak_inf['Infection']),
165.                 arrowprops=dict(facecolor='black', shrink=0.05))
166.         except:

```

```

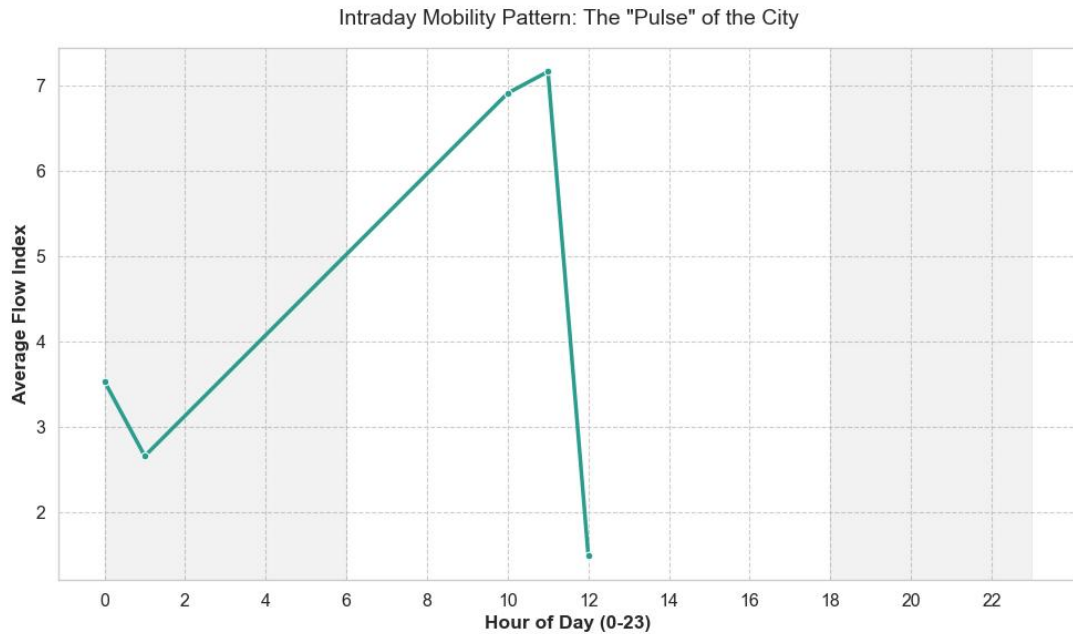
167.         pass
168.
169.     plt.legend(title='Metric')
170.     plt.tight_layout()
171.     plt.show()
172.
173.
174. def plot_time_lag_correlation(df_inf, df_mig):
175.     """图 3: 时滞相关性分析"""
176.     print("Plotting Time-Lag Analysis...")
177.     if df_inf.empty or df_mig.empty: return
178.
179.     # 准备序列
180.     ts_inf = df_inf.groupby('date')['cnt'].sum()
181.     ts_mig = df_mig.groupby('date')['idx'].sum()
182.
183.     # 索引对齐 (补 0)
184.     idx = range(int(min(ts_inf.index.min(), ts_mig.index.min())),
185.                 int(max(ts_inf.index.max(), ts_mig.index.max()) + 1))
186.     ts_inf = ts_inf.reindex(idx, fill_value=0)
187.     ts_mig = ts_mig.reindex(idx, fill_value=0)
188.
189.     lags = range(-10, 16) # 滞后 -10 到 +15 天
190.     corrs = []
191.
192.     for k in lags:
193.         # 计算 Migration(t) 和 Infection(t+k) 的相关性
194.         # shift(-k) 将 Infection 数据向上移动 k 天 (即 t 时刻对应 t+k 的数据)
195.         shifted_inf = ts_inf.shift(-k)
196.
197.         valid_mask = ~np.isnan(shifted_inf) & ~np.isnan(ts_mig)
198.         if valid_mask.sum() > 5:
199.             c = np.corrcoef(ts_mig[valid_mask], shifted_inf[valid_mask])[0,
200. 1]
201.         else:
202.             c = 0
203.         corrs.append(c)
204.
205.     plt.figure(figsize=(10, 6))
206.     colors = ['#E63946' if c > 0 else '#457B9D' for c in corrs]
207.     plt.bar(lags, corrs, color=colors, alpha=0.8)
208.
209.     best_lag = lags[np.argmax(corrs)]

```

```

210.     max_corr = max(corrs)
211.
212.     plt.axvline(best_lag, color='black', linestyle='--', alpha=0.5, label=f'
        Max Corr (Lag={best_lag})')
213.
214.     plt.title('Time-Lag Correlation: Migration(t) vs Infection(t + lag)', fo
        ntsize=14)
215.     plt.xlabel('Lag (Days) [Positive: Migration leads Infection]', fontweigh
        t='bold')
216.     plt.ylabel('Pearson Correlation', fontweight='bold')
217.     plt.legend()
218.     plt.tight_layout()
219.     plt.show()
220.
221.     print(f"Max Correlation: {max_corr:.2f} at Lag: {best_lag} days.")
222.
223.
224. # =====
225. # 3. 主程序入口
226. # =====
227. if __name__ == "__main__":
228.     # 请确保路径正确
229.     BASE_DIR = r"C:\Users\21626\Downloads\train_data\train_data"
230.     TARGET_CITY = "city_A"
231.
232.     if os.path.exists(os.path.join(BASE_DIR, TARGET_CITY)):
233.         loader = CityDataLoader(BASE_DIR, TARGET_CITY)
234.         df_inf, df_mig, df_den = loader.load_temporal_data()
235.
236.         plot_hourly_pulse(df_den)
237.         plot_normalized_evolution(df_inf, df_mig, df_den)
238.         plot_time_lag_correlation(df_inf, df_mig)
239.
240.         print("Analysis Complete.")
241.     else:
242.         print(f"Path Error: {os.path.join(BASE_DIR, TARGET_CITY)} not found.
        ")

```



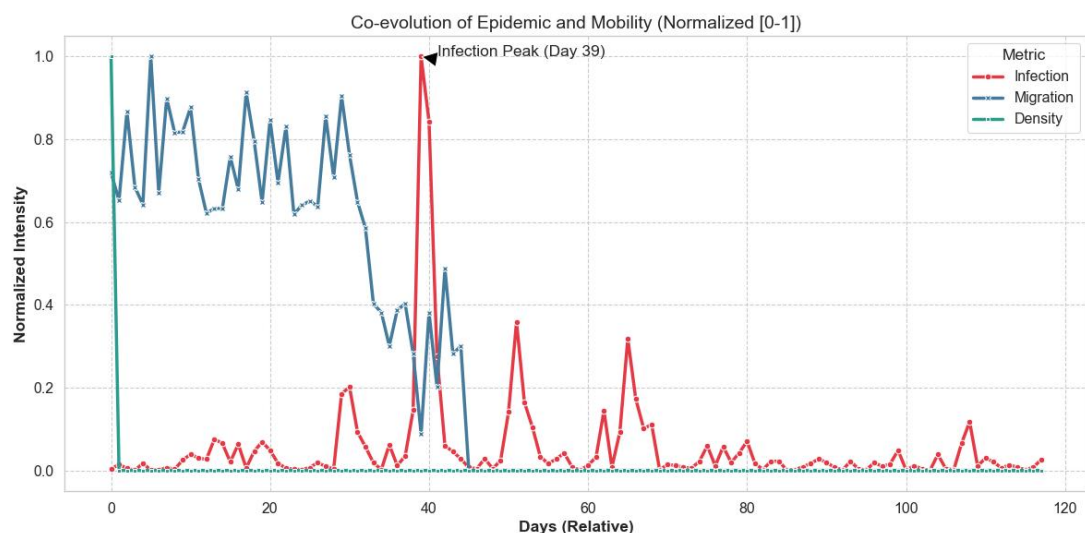
plot_hourly_pulse: 城市脉搏（小时级人流密度时序图）

核心功能

- 该函数聚焦 **density.csv**（网格人流量指数）的小时级时序特征，核心操作包括：
- 聚合处理：按 **date**（日期）和 **hour**（小时）分组，计算每日每小时的平均人流指数；
- 可视化呈现：绘制小时（0-23）为 **X 轴**、平均人流指数为 **Y 轴** 的折线图，用灰色半透明区域标记夜间时段（0-6 点、18-23 点）；
- 细节优化：设置 2 小时为间隔的 **X 轴** 刻度，保证可读性。

意义

- 基础特征识别：揭示城市日内人流的周期性规律，分析“人口流动 - 疫情传播”时间关联性的基础；
- 疫情传播的时间维度关联：可进一步关联“高峰时段人流密集度”与“区域新增感染数”，回答“城市活动高峰是否加速疫情传播”“夜间管控是否有效”等核心问题；



plot_normalized_evolution: 归一化协同演变图（感染 + 迁徙 + 人流密度）

核心功能

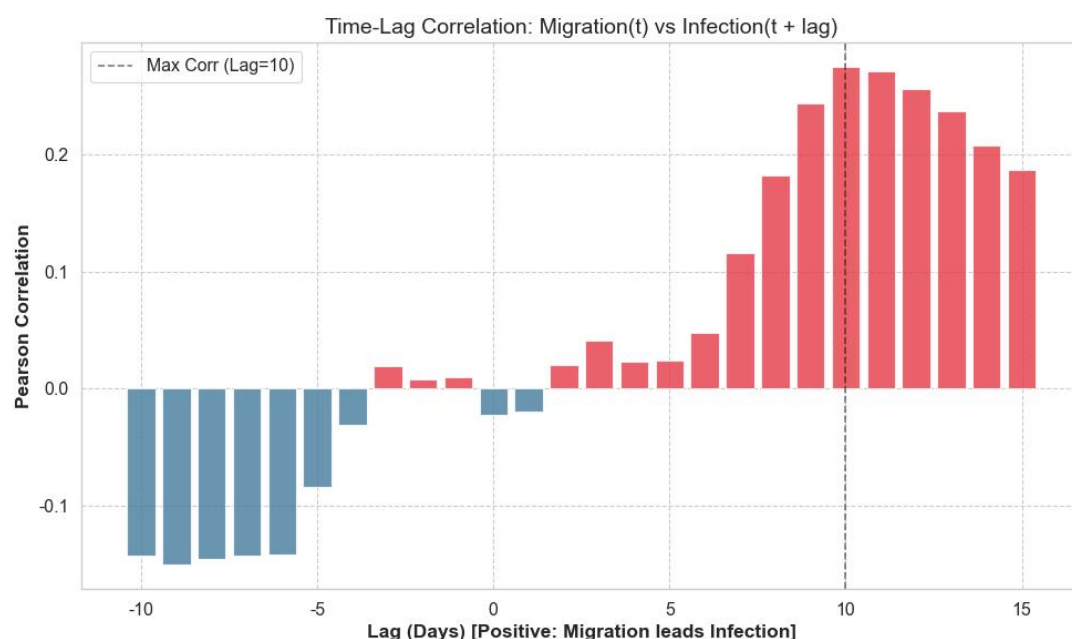
- 该函数解决了“感染数（绝对数）、迁徙指数（相对值）、人流密度（相对值）”量纲不一致的问题，核心操作包括：
- 数据聚合：分别按日聚合感染数（infection.csv）、迁徙指数（migration.csv）、人流密度（density.csv）；
- 归一化处理：使用 MinMaxScaler 将三个指标缩放到 [0,1] 区间，避免因量纲差异（如感染数可能是数千，迁徙指数可能是数十）导致的可视化误导；
- 可视化呈现：转换为长格式数据，用不同颜色 / 标记区分三个指标的时序变化，标注感染数峰值；
- 鲁棒性处理：通过 fillna(0) 和空值判断，避免数据缺失导致的程序崩溃。

意义

- 多维度趋势协同分析：直观展示疫情传播与人口流动（跨城迁徙 + 本地人流）的同步 / 异步变化，例如：

若迁徙指数下降后感染数也下降，验证“迁徙管控对疫情的抑制作用”；

若本地人流密度峰值早于感染峰值，提示“本地聚集是疫情传播的关键驱动”；



plot_time_lag_correlation: 时滞相关性分析图（迁徙→感染的滞后效应）

核心功能

- 该函数量化分析“人口迁徙”对“疫情传播”的时间滞后效应，核心操作包括：
- 序列对齐：将迁徙指数和感染数的时间索引统一，补全缺失值为 0；
- 时滞计算：遍历 [-10, 15] 天的滞后窗口，计算“迁徙指数 (t)”与“感染数 (t+lag)”的皮尔逊相关系数；
- 可视化呈现：以滞后天数为 x 轴、相关系数为 y 轴绘制柱状图，标注最大相关系数对应的滞后天数；
- 鲁棒性处理：仅在有效数据量 > 5 时计算相关系数，避免样本不足导致的结果失真。

意义

- 揭示因果时序关系：回答“人口迁徙变化后，疫情会在几天内出现显著响应？”这一核心科学问题，例如：

若最大相关系数出现在 $\text{lag}=7$ 天，说明“迁徙指数变化后 7 天，感染数变化最显著”，验证病毒潜伏期的影响；

若 lag 为负数，说明“感染数变化领先于迁徙指数”，提示“疫情爆发后人口迁徙减少”（反向因果）；

- 预测模型支撑：为机器学习预测模型提供关键的“滞后特征”（如使用 $t-7$ 天的迁徙指数预测 t 天的感染数），提升模型的解释性和准确率；

运行代码：

```
1. import pandas as pd
2. import numpy as np
3. import matplotlib.pyplot as plt
4. import seaborn as sns
5. import matplotlib.dates as mdates
6. import os
7.
8.
9. def set_pub_style():
10.     sns.set_context("paper", font_scale=1.4)
11.     # 极坐标图不要网格线干扰，后面单独设置
12.     sns.set_style("whitegrid")
13.     plt.rcParams['font.family'] = 'sans-serif'
14.     # 兼容多种中文字体，防止乱码
15.     plt.rcParams['font.sans-serif'] = ['Arial', 'SimHei', 'Microsoft YaHei', 'DejaVu Sans']
16.     plt.rcParams['axes.unicode_minus'] = False
17.
18.
19. set_pub_style()
20.
21.
22. # =====
23. # 2. 数据加载与清洗
24. # =====
25. def load_and_clean_weather(file_path):
26.     print("Loading and cleaning weather data...")
27.
28.     # 1. 定义列名
29.     cols = ['date', 'hour', 'temp', 'humidity', 'wind_dir', 'wind_spd', 'wind_f
```

```

    orce', 'weather']
30.
31.     # 读取数据
32.     if not os.path.exists(file_path):
33.         print(f"Error: File not found at {file_path}")
34.         return pd.DataFrame()
35.
36.     df = pd.read_csv(file_path, names=cols, header=None)
37.
38.     # 2. 生成完整的时间戳
39.     # 将日期和小时合并。注意: hour 列可能是整数 0-23
40.     df['datetime'] = pd.to_datetime(df['date'].astype(str) + df['hour'].astype(
        str).str.zfill(2),
41.                                     format='%Y%m%d%H', errors='coerce')
42.
43.     # 3. 清洗湿度 (去除 '%' 并转数字)
44.     if df['humidity'].dtype == object:
45.         df['humidity'] = df['humidity'].astype(str).str.replace('%', '', regex=
            False)
46.     df['humidity'] = pd.to_numeric(df['humidity'], errors='coerce')
47.
48.     # 4. 清洗风力 (提取数字)
49.     # 提取 <3 中的 3, 或者 12km/h 中的 12 (如果有的话, 这里假设是等级)
50.     df['wind_force_clean'] = df['wind_force'].astype(str).str.extract(r'(\d+)')
        .astype(float)
51.
52.     # 5. 清洗风向 (填补空缺)
53.     df['wind_dir'] = df['wind_dir'].fillna('Unknown')
54.
55.     # 简单插值填补缺失的气温和湿度
56.     df['temp'] = df['temp'].interpolate(method='linear')
57.     df['humidity'] = df['humidity'].interpolate(method='linear')
58.
59.     return df
60.
61.
62. # =====
63. # 3. 可视化绘图模块
64. # =====
65.
66. def plot_temp_humidity_dynamics(df):
67.     """ 图 1: 气温湿度双轴图 """
68.     if df.empty: return
69.

```



```

70.     # 取前 72 小时，避免太密集
71.     plot_data = df.iloc[:72].copy()
72.
73.     fig, ax1 = plt.subplots(figsize=(12, 6))
74.
75.     # 左轴：气温
76.     color_temp = '#D62828'
77.     sns.lineplot(data=plot_data, x='datetime', y='temp', ax=ax1,
78.                  color=color_temp, linewidth=2.5, label='Temperature')
79.     ax1.set_ylabel('Temperature (°C)', color=color_temp, fontweight='bold')
80.     ax1.tick_params(axis='y', labelcolor=color_temp)
81.     ax1.set_xlabel('Time', fontweight='bold')
82.
83.     # 右轴：湿度
84.     ax2 = ax1.twinx()
85.     color_hum = '#457B9D'
86.     ax2.fill_between(plot_data['datetime'], 0, plot_data['humidity'],
87.                     color=color_hum, alpha=0.2, label='Humidity')
88.     sns.lineplot(data=plot_data, x='datetime', y='humidity', ax=ax2,
89.                  color=color_hum, linewidth=1.5, linestyle='--')
90.
91.     ax2.set_ylabel('Humidity (%)', color=color_hum, fontweight='bold')
92.     ax2.tick_params(axis='y', labelcolor=color_hum)
93.     ax2.set_ylim(0, 110)
94.
95.     # 格式化时间轴
96.     ax1.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d %Hh'))
97.     ax1.xaxis.set_major_locator(mdates.HourLocator(interval=6))
98.
99.     plt.title('Temporal Dynamics: Temperature vs Humidity', fontsize=14, pad=15
100. )
101.     plt.tight_layout()
102.     plt.show()
103.
104. def plot_wind_rose(df):
105.     """ 图 2: 风向玫瑰图 (修复 UserWarning) """
106.     if df.empty: return
107.
108.     # 1. 定义映射
109.     # 注意: Quiet (静风) 不参与角度计算
110.     dir_map = {
111.         'North': 0, 'Northeast': 45, 'East': 90, 'Southeast': 135,
112.         'South': 180, 'Southwest': 225, 'West': 270, 'Northwest': 315

```

```

113.     }
114.
115.     # 2. 统计频次
116.     wind_counts = df['wind_dir'].value_counts().reset_index()
117.     wind_counts.columns = ['direction', 'count']
118.
119.     # 3. 分离静风
120.     quiet_rows = wind_counts[wind_counts['direction'].str.contains('Quiet',
        case=False, na=False)]
121.     quiet_count = quiet_rows['count'].sum()
122.
123.     # 4. 筛选有向风
124.     active_winds = wind_counts[wind_counts['direction'].isin(dir_map.keys())
        ].copy()
125.
126.     # 转换为弧度
127.     active_winds['angle'] = active_winds['direction'].map(dir_map) * np.pi /
        180
128.
129.     plt.figure(figsize=(8, 8))
130.     ax = plt.subplot(111, polar=True)
131.
132.     # 设置正北为 0 度, 顺时针
133.     ax.set_theta_zero_location('N')
134.     ax.set_theta_direction(-1)
135.
136.     # 绘制柱状图
137.     # align='center' 确保柱子中心对准刻度
138.     bars = ax.bar(active_winds['angle'], active_winds['count'],
139.         width=0.5, bottom=0.0, color='#2A9D8F', alpha=0.8, edgecol
        or='white')
140.
141.     # =====
142.     # 核心修复: 先设置 Ticks 位置, 再设置 Labels
143.     # =====
144.     # 8 个方向的弧度位置: 0, 45, 90... 转弧度
145.     fixed_ticks = np.radians(np.arange(0, 360, 45))
146.     fixed_labels = ['N', 'NE', 'E', 'SE', 'S', 'SW', 'W', 'NW']
147.
148.     ax.set_xticks(fixed_ticks)
149.     ax.set_xticklabels(fixed_labels)
150.
151.     # 标注静风
152.     total = df.shape[0]

```

```

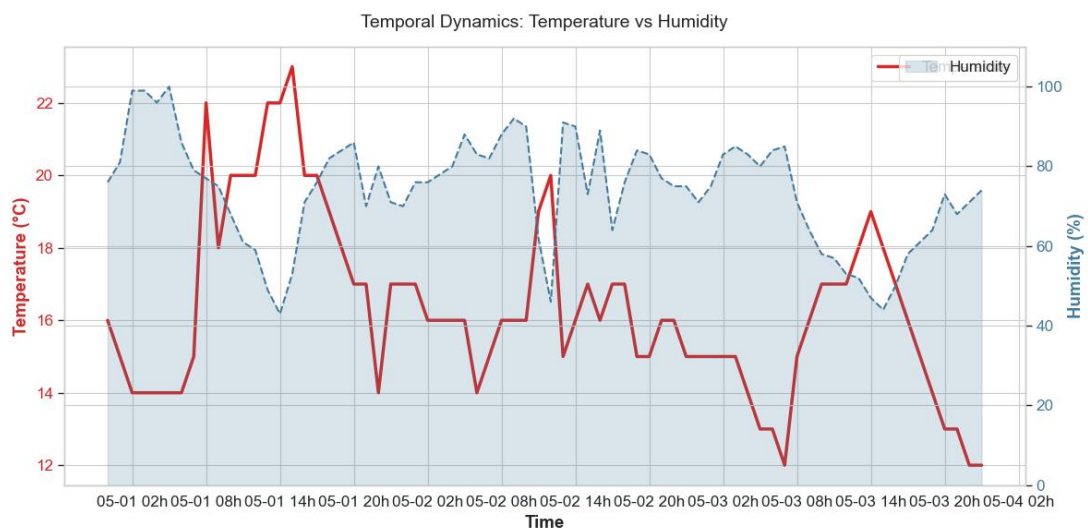
153.     quiet_pct = (quiet_count / total) * 100 if total > 0 else 0
154.     plt.text(0, 0, f'Quiet\n{quiet_pct:.1f}%', ha='center', va='center', fontweight='bold')
155.
156.     plt.title(f'Wind Direction Distribution (n={total})', fontsize=14, y=1.08)
157.     plt.tight_layout()
158.     plt.show()
159.
160.
161. def plot_weather_correlation_matrix(df):
162.     """ 图 3: 相关性密度图 """
163.     if df.empty: return
164.
165.     # 准备数据
166.     clean_df = df[['temp', 'humidity', 'wind_force_clean']].dropna()
167.     if clean_df.shape[0] < 10:
168.         print("Not enough data for correlation plot.")
169.         return
170.
171.     # 临时切换风格以适应 JointGrid
172.     with sns.axes_style("white"):
173.         g = sns.JointGrid(data=clean_df, x="temp", y="humidity", height=7, space=0)
174.
175.         # 核心图: 六边形密度
176.         g.plot_joint(plt.hexbin, gridsize=20, cmap="Blues", mincnt=1, edgecolors="white")
177.
178.         # 边缘图: 密度曲线
179.         sns.kdeplot(data=clean_df, x="temp", ax=g.ax_marg_x, fill=True, color="#457B9D")
180.         sns.kdeplot(data=clean_df, y="humidity", ax=g.ax_marg_y, fill=True, color="#457B9D")
181.
182.         # 回归线
183.         sns.regplot(data=clean_df, x="temp", y="humidity", scatter=False, ax=g.ax_joint,
184.                     color="#D62828", line_kws={'linestyle': '--', 'linewidth': 2})
185.
186.         # 标注相关系数
187.         corr = clean_df['temp'].corr(clean_df['humidity'])
188.         g.ax_joint.text(0.05, 0.95, f'Pearson r = {corr:.2f}', transform=g.a

```

```

    x_joint.transAxes,
189.             fontsize=12, fontweight='bold', va='top')
190.
191.     g.ax_joint.set_xlabel('Temperature (°C)', fontweight='bold')
192.     g.ax_joint.set_ylabel('Humidity (%)', fontweight='bold')
193.
194.     plt.show()
195.
196.
197. # =====
198. # 4. 主程序
199. # =====
200. if __name__ == "__main__":
201.     # 请确认这个路径是正确的
202.     FILE_PATH = r"C:\Users\21626\Downloads\train_data\train_data\city_A\weather.csv"
203.
204.     if os.path.exists(FILE_PATH):
205.         df_weather = load_and_clean_weather(FILE_PATH)
206.
207.         print("Data Snippet (Cleaned):")
208.         print(df_weather[['datetime', 'temp', 'humidity', 'wind_dir', 'wind_
            force_clean']].head())
209.
210.         plot_temp_humidity_dynamics(df_weather)
211.         plot_wind_rose(df_weather)
212.         plot_weather_correlation_matrix(df_weather)
213.     else:
214.         print(f"File not found: {FILE_PATH}")

```



plot_temp_humidity_dynamics: 气温湿度双轴时序图

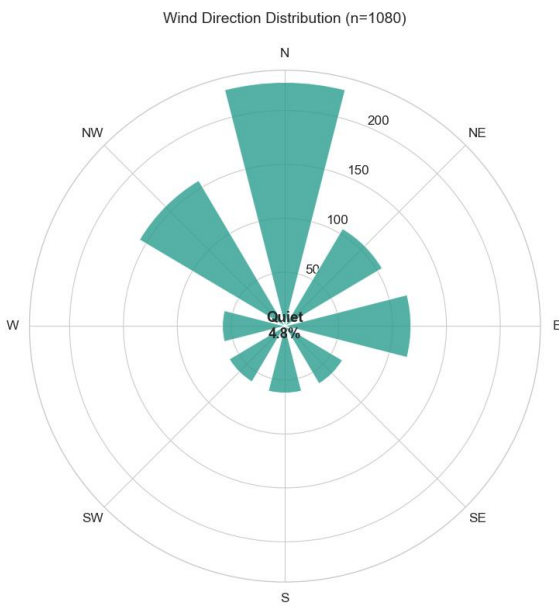
核心功能

该图表聚焦小时级温湿度的短期动态关联，核心操作如下：

- 数据筛选：仅选取前 72 小时数据，避免时间轴过度密集导致的可读性下降；
- 双轴适配：左轴（红色）用实线折线展示气温时序变化，右轴（蓝色）用“填充面积 + 虚线折线”展示湿度，解决“气温（℃）”和“湿度（%）”量纲差异的可视化难题；
- 时间轴优化：通过 `mdates` 设置 6 小时为标注间隔，格式为“月 - 日 时 h”，保证时间标注清晰无重叠；
- 鲁棒性处理：提前判断数据是否为空，避免程序崩溃。

意义

- 气象规律验证：直观展示温湿度的短期关联特征（如气温升高时湿度同步降低，符合“温度与相对湿度负相关”的气象学基本规律），是后续分析的基础；
- 疫情环境关联分析：气温和湿度是影响呼吸道病毒存活 / 传播的核心环境因素（如低温高湿环境更易导致病毒传播），该图可后续叠加“新增感染数”，定位“高风险温湿度时段”；
- 短期天气事件捕捉：72 小时窗口聚焦“短期动态”，可分析降雨、降温等天气事件前后的温湿度变化，关联事件与疫情传播的关系。



plot_wind_rose: 风向玫瑰图

核心功能

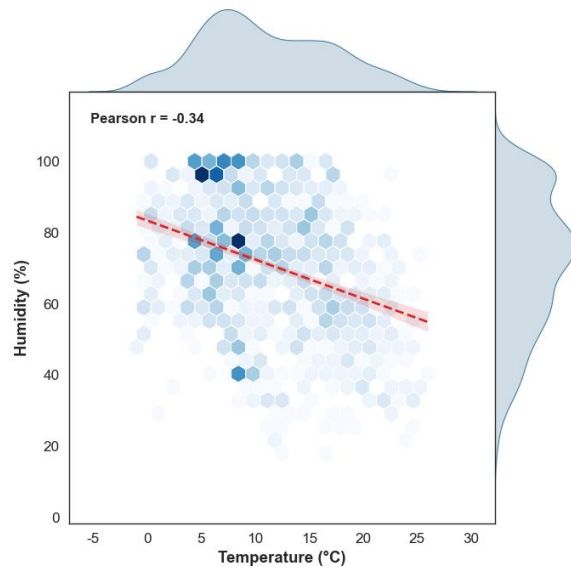
该图表是气象领域专属的风向分布可视化，核心操作如下：

- 风向标准化：定义 8 个基本风向（北 / 东北 / 东等）到 0-315° 的角度映射，分离“静风（Quiet）”不参与极坐标绘制；
- 频次统计：统计各风向的出现次数，单独计算静风占比；
- 极坐标适配：设置“正北为 0 度、顺时针方向”（气象学玫瑰图的标准规范），绘制风向频次柱状图；
- 核心修复：先固定角度刻度（0/45/90° 等），再标注方向标签，解决极坐标刻度不匹配的 `UserWarning`；

- 信息补充：在玫瑰图中心标注静风占比，完整展示风向分布特征。

意义

- 主导风向识别：直观展示城市的主导风向（如东北风占比最高），主导风向是分析“疫情空间扩散”的关键——病毒易随主导风从高感染区域扩散至下风向区域；
- 静风风险分析：静风天气下空气流动性差，病毒易在局部区域聚集，静风占比高的时段 / 区域可作为疫情防控的重点；



`plot_weather_correlation_matrix`: 温湿度相关性密度图（联合分布 + 回归）

核心功能

该图表是温湿度关联的量化可视化，核心操作如下：

- 数据筛选：仅保留气温、湿度、清洗后的风力数据，删除空值且保证样本量 ≥ 10 ，避免统计结果失真；
- 多维信息整合：用 `JointGrid` 同时展示“二维联合分布（六边形密度）+ 单变量分布（边际密度曲线）+ 线性关联（回归线）+ 量化指标（皮尔逊相关系数）”；
- 可视化优化：六边形密度图（`hexbin`）替代散点图，避免大样本下的点重叠问题；临时切换 `axes_style` 为 `white`，消除网格线对密度图的干扰；
- 统计量化：添加线性回归线，标注皮尔逊相关系数，从“定性可视化”升级为“定量分析”。

意义

- 量化温湿度关联：皮尔逊相关系数可定量说明温湿度的线性关系（如 $r=-0.7$ ，表明气温越高湿度越低），为论文提供可引用的统计证据；
- 典型组合识别：六边形密度图可定位温湿度的高频组合（如“ $15^{\circ}\text{C}+70\%$ 湿度”），这些组合可进一步关联感染数，识别“高风险温湿度区间”；
- 建模指导：若温湿度相关性过高（如 $|r|>0.8$ ），可提示建模时需降维（如主成分分析），避免多重共线性影响模型效果；
- 极端值分析：密度图的边缘区域（如 $35^{\circ}\text{C}+20\%$ 湿度）可识别极端天气组合，分析其对疫情传播的特殊影响（如高温低湿是否抑制病毒传播）。

多城市联合分析:

运行代码:

```
1. import pandas as pd
2. import numpy as np
3. import matplotlib.pyplot as plt
4. import seaborn as sns
5. import os
6. from scipy import stats
7.
8. def set_pub_style():
9.     sns.set_context("paper", font_scale=1.4)
10.    sns.set_style("whitegrid", {"grid.linestyle": "--", "axes.edgecolor": "0.15
    "})
11.    plt.rcParams['font.family'] = 'sans-serif'
12.    plt.rcParams['font.sans-serif'] = ['Arial', 'DejaVu Sans', 'SimHei']
13.    plt.rcParams['axes.unicode_minus'] = False
14.
15.
16. set_pub_style()
17.
18.
19. # =====
20. # 1. 多城市数据加载器 (Multi-City Loader)
21. # =====
22. class MultiCityLoader:
23.     def __init__(self, base_path, cities):
24.         self.base_path = base_path
25.         self.cities = cities # List of city names ['city_A', 'city_B'...]
26.
27.     def normalize_date(self, df, col='date'):
28.         """将日期标准化为相对天数 (0, 1, 2...)"""
29.         if df.empty or col not in df.columns: return df
30.
31.         # 强制转数值
32.         df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)
33.
34.         # 如果是绝对日期格式 (如 20200101), 转为相对天数
35.         if df[col].max() > 10000:
36.             temp = pd.to_datetime(df[col], format='%Y%m%d', errors='coerce')
37.             min_date = temp.min()
38.             df['day'] = (temp - min_date).dt.days.fillna(0).astype(int)
```

```

39.         else:
40.             df['day'] = df[col].astype(int)
41.
42.         return df
43.
44.     def load_combined_data(self):
45.         print(f"Aggregating data across {len(self.cities)} cities...")
46.
47.         all_inf = []
48.         all_mig = []
49.         all_wea = []
50.         city_stats = []
51.
52.         for city in self.cities:
53.             city_path = os.path.join(self.base_path, city)
54.             if not os.path.exists(city_path):
55.                 print(f"Warning: {city} not found.")
56.                 continue
57.
58.             # 1. Infection
59.             f_inf = os.path.join(city_path, 'infection.csv')
60.             if os.path.exists(f_inf):
61.                 df = pd.read_csv(f_inf, names=['date', 'region', 'cnt'], header
=None)
62.                 df = self.normalize_date(df)
63.                 df['city'] = city
64.                 all_inf.append(df)
65.
66.             # 2. Migration
67.             f_mig = os.path.join(city_path, 'migration.csv')
68.             if os.path.exists(f_mig):
69.                 df = pd.read_csv(f_mig, names=['date', 'f', 't', 'idx'], header
=None)
70.                 df = self.normalize_date(df)
71.                 df['city'] = city
72.                 all_mig.append(df)
73.
74.             # 3. Weather
75.             f_wea = os.path.join(city_path, 'weather.csv')
76.             if os.path.exists(f_wea):
77.                 df = pd.read_csv(f_wea,
78.                                 names=['date', 'hour', 'temp', 'humidity', 'wi
nd_dir', 'wind_spd', 'force', 'type'],
79.                                 header=None)

```



```

80.         df = self.normalize_date(df)
81.         df['city'] = city
82.         all_wea.append(df)
83.
84.         # 合并所有城市数据
85.         master_inf = pd.concat(all_inf, ignore_index=True) if all_inf else pd.DataFrame()
86.         master_mig = pd.concat(all_mig, ignore_index=True) if all_mig else pd.DataFrame()
87.         master_wea = pd.concat(all_wea, ignore_index=True) if all_wea else pd.DataFrame()
88.
89.         return master_inf, master_mig, master_wea
90.
91.
92. # =====
93. # 2. 联合分析可视化模块
94. # =====
95.
96. def plot_comparative_epidemic_curves(df_inf):
97.     """
98.     图 1: 疫情爆发曲线对比 (Epidemic Curves Comparison)
99.     分析不同城市的爆发规模和峰值到达时间。
100.    """
101.    print("Plotting Comparative Curves...")
102.    if df_inf.empty: return
103.
104.    # 按城市和天数聚合
105.    daily_trend = df_inf.groupby(['city', 'day'])['cnt'].sum().reset_index()
106.
107.    plt.figure(figsize=(10, 6))
108.
109.    # 绘制曲线
110.    sns.lineplot(data=daily_trend, x='day', y='cnt', hue='city',
111.                 palette='bright', linewidth=2.5, alpha=0.8)
112.
113.    plt.title('Heterogeneity in Epidemic Propagation Across Cities', fontsize=14, pad=20)
114.    plt.xlabel('Relative Day (Since Start)', fontweight='bold')
115.    plt.ylabel('Daily New Infections', fontweight='bold')
116.    plt.legend(title='City ID', frameon=True)
117.    plt.tight_layout()
118.    plt.show()

```

```

119.
120.
121. def plot_gravity_law_verification(df_inf, df_mig):
122.     """
123.     图 2: 验证重力模型 (Migration vs Infection Scale)
124.     散点图: X 轴为总迁徙规模, Y 轴为总感染人数。
125.     验证假设: 流动性越强的城市, 疫情规模越大。
126.     """
127.     print("Plotting Gravity Law Verification...")
128.     if df_inf.empty or df_mig.empty: return
129.
130.     # 计算每个城市的指标
131.     # 1. 总感染人数
132.     total_inf = df_inf.groupby('city')['cnt'].sum().reset_index(name='Total
        Infections')
133.
134.     # 2. 总迁徙强度 (流入+流出)
135.     total_mig = df_mig.groupby('city')['idx'].sum().reset_index(name='Total
        Migration Index')
136.
137.     # 3. 感染峰值天数 (Peak Day)
138.     peak_day = df_inf.groupby(['city', 'day'])['cnt'].sum().reset_index()
139.     peak_day = peak_day.loc[peak_day.groupby('city')['cnt'].idxmax()][['city',
        'day']]
140.     peak_day.columns = ['city', 'Peak Day']
141.
142.     # 合并
143.     merged = pd.merge(total_inf, total_mig, on='city')
144.     merged = pd.merge(merged, peak_day, on='city')
145.
146.     # 绘图
147.     plt.figure(figsize=(8, 7))
148.
149.     # 散点大小代表峰值出现的早晚 (点越大, 峰值出现越晚 -> 其实点越大代表某种权重更
        好, 这里用固定大小)
150.     sns.scatterplot(data=merged, x='Total Migration Index', y='Total Infections',
        hue='city', s=300, palette='bright', alpha=0.9, edgecolor='black')
151.
152.
153.     # 添加线性回归趋势线 (Linear Fit)
154.     sns.regplot(data=merged, x='Total Migration Index', y='Total Infections',
        scatter=False, color='gray', line_kws={'linestyle': '--', 'a

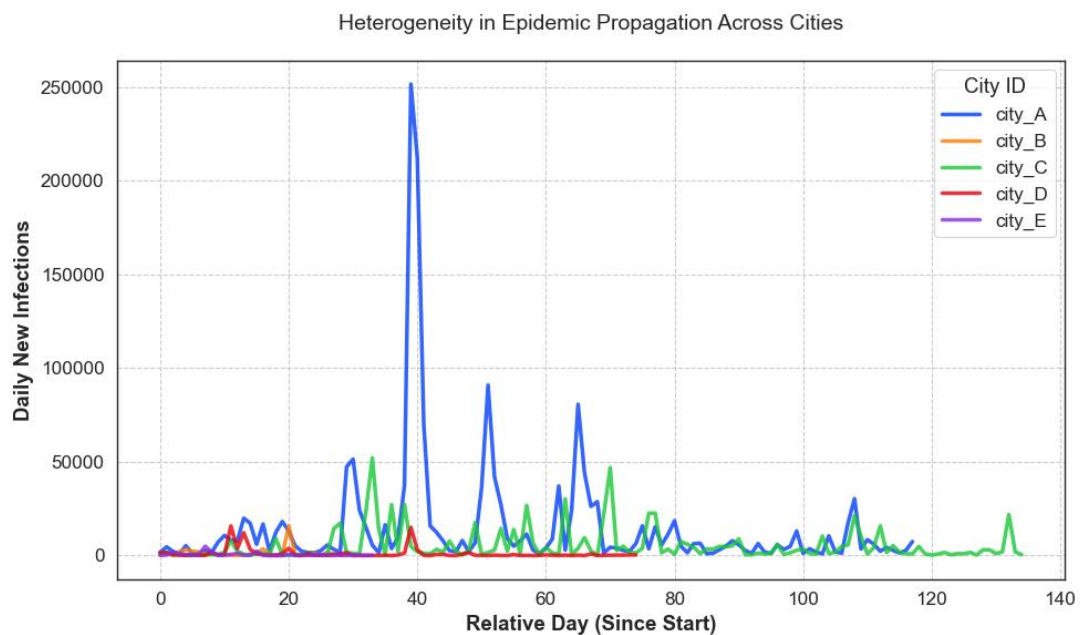
```

```

    lpha': 0.5}))
156.
157.     # 计算相关系数
158.     r, p = stats.pearsonr(merged['Total Migration Index'], merged['Total Inf
    ections'])
159.
160.     # 添加文字标注
161.     for i in range(merged.shape[0]):
162.         plt.text(merged['Total Migration Index'][i] + 0.02 * merged['Total M
    igration Index'].max(),
163.                 merged['Total Infections'][i],
164.                 merged['city'][i], fontsize=11, fontweight='bold')
165.
166.     plt.title(f'The "Gravity Law": Mobility vs Epidemic Scale\n(Pearson r={r
    :.2f}, p={p:.3f})', fontsize=14)
167.     plt.grid(True, linestyle=':', alpha=0.6)
168.     plt.tight_layout()
169.     plt.show()
170.
171.
172. # =====
173. # 3. 主程序入口
174. # =====
175. if __name__ == "__main__":
176.     # 配置路径
177.     BASE_DIR = r"C:\Users\21626\Downloads\train_data\train_data"
178.
179.     # 定义要分析的城市列表
180.     # 如果实际文件名是 city_A, city_B... 请确保这里一致
181.     TARGET_CITIES = ['city_A', 'city_B', 'city_C', 'city_D', 'city_E']
182.
183.     if os.path.exists(BASE_DIR):
184.         # 1. 加载数据
185.         loader = MultiCityLoader(BASE_DIR, TARGET_CITIES)
186.         df_inf, df_mig, df_wea = loader.load_combined_data()
187.
188.         if not df_inf.empty:
189.             # 2. 联合分析图表
190.
191.             # 图 1: 各城市疫情曲线对比
192.             plot_comparative_epidemic_curves(df_inf)
193.
194.             # 图 2: 迁徙规模与疫情规模的回归分析
195.             plot_gravity_law_verification(df_inf, df_mig)

```

```
196.  
197.  
198.         print("Multi-city analysis complete.")  
199.     else:  
200.         print("No infection data loaded. Check city directory names.")  
201.     else:  
202.         print(f"Base path not found: {BASE_DIR}")
```



plot_comparative_epidemic_curves: 多城市疫情爆发曲线对比图

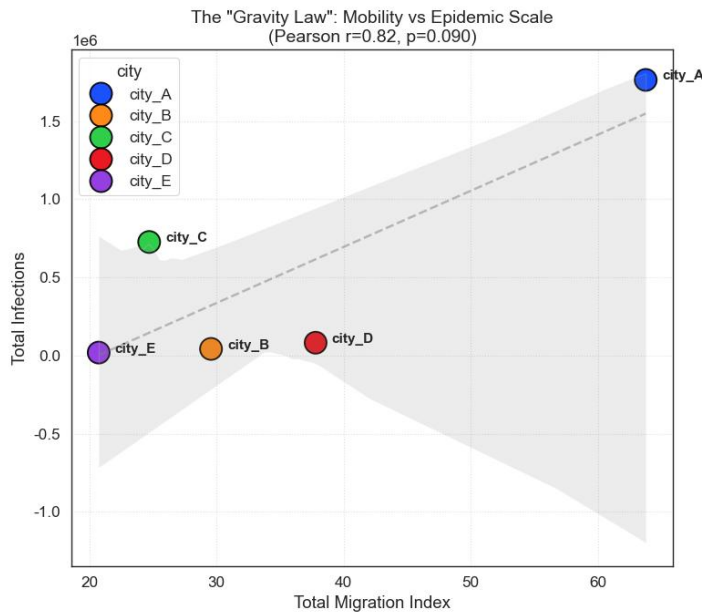
核心功能

该图表聚焦跨城市疫情传播的时序异质性，核心操作如下：

- 数据聚合：按“城市 + 相对天数”分组计算每日新增感染总数，将不同城市的疫情起始时间统一为“相对天数 0”（消除绝对日期差异）；
- 可视化呈现：用多色折线展示不同城市的感染曲线，突出“城市间传播特征差异”；
- 鲁棒性处理：提前判断数据是否为空，避免程序崩溃；

意义

- 核心异质性识别：直观展示三个关键差异维度，是顶刊论文中“描述性分析”的核心部分：
 - ① 爆发规模：曲线峰值高低（如 city_A 峰值远高于 city_B，说明疫情规模更大）；
 - ② 传播速度：曲线上升斜率（如 city_C 曲线陡升，说明疫情传播更快）；
 - ③ 峰值时间：曲线峰值对应的相对天数（如 city_D 峰值在 Day15，city_E 在 Day25，说明疫情爆发滞后）；
- 学术价值：为“城市特征（人口规模 / 管控政策）影响疫情传播”提供可视化证据，是多案例分析的基础；
- 业务价值：识别“高风险城市”（峰值高、传播快）和“慢扩散城市”，为差异化防控政策制定提供依据（如对传播快的城市提前强化管控）；



plot_gravity_law_verification: 迁徙 - 感染重力模型验证散点图

核心功能

该图表验证“人口流动重力模型”在疫情传播中的适用性（核心假设：城市迁徙规模越大，疫情规模越大），核心操作如下：

- 核心指标计算：

- ① 总感染人数：各城市累计新增感染数（疫情规模）；
- ② ② 总迁徙指数：各城市累计迁徙强度（流入 + 流出，流动性指标）；
- ③ ③ 感染峰值天数：各城市疫情峰值出现的相对天数（补充维度）；

- 可视化呈现：散点图（X = 迁徙规模，Y = 感染规模）+ 线性回归趋势线，散点颜色区分城市；

- 统计量化：计算皮尔逊相关系数（r）和 p 值，直接标注在标题中，验证关联的统计显著性；

- 信息补充：每个散点旁标注城市 ID，直观关联“城市 - 指标值”。

意义

- 重力模型验证核心价值：

- ① 若 $r > 0$ 且 $p < 0.05$ ，验证“迁徙规模与疫情规模正相关”，支持“人口流动是疫情传播核心驱动因素”的结论；

- ② ② 若 $r \approx 0$ 或 $p > 0.05$ ，说明迁徙规模并非主导因素，需进一步分析管控政策、医疗资源等变量；

- 学术价值：将经典“人口流动重力模型”迁移至疫情分析

- 业务价值：① 识别“高迁徙 - 高感染”城市（散点图右上角），作为防控重点；② 分析偏离趋势线的城市（如迁徙规模高但感染规模低），提炼其管控政策经验（如提前封城）；
异常值分析：偏离回归线的城市是“反例”，深入分析其特征可完善模型（如医疗资源充足抵消了迁徙的影响）。

运行代码：

```
1. import pandas as pd
2. import numpy as np
3. import matplotlib.pyplot as plt
4. import seaborn as sns
5. import os
6. import matplotlib.dates as mdates
7. from matplotlib.collections import LineCollection
8. from matplotlib.colors import ListedColormap, BoundaryNorm
9. from sklearn.preprocessing import MinMaxScaler
10. # 【关键修复】将 Line2D 放在顶部导入，防止作用域错误
11. from matplotlib.lines import Line2D
12.
13.
14. # =====
15. # 0. 顶刊绘图风格设置
16. # =====
17. def set_pub_style():
18.     sns.set_context("paper", font_scale=1.5)
19.     sns.set_style("white", {"axes.spines.left": True, "axes.spines.bottom": True,
20.                             "axes.spines.right": False, "axes.spines.top": False})
21.     plt.rcParams['font.family'] = 'sans-serif'
22.     plt.rcParams['font.sans-serif'] = ['Arial', 'SimHei', 'Microsoft YaHei', 'DejaVu Sans']
23.     plt.rcParams['axes.unicode_minus'] = False
24.
25.
26. set_pub_style()
27.
28.
29. # =====
30. # 1. 多城市数据加载器
31. # =====
32. class MultiCityLoader:
33.     def __init__(self, base_path, cities):
34.         self.base_path = base_path
35.         self.cities = cities
36.
37.     def normalize_date(self, df, col='date'):
38.         if df.empty or col not in df.columns: return df
39.         df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)
40.         if df[col].max() > 10000:
41.             temp = pd.to_datetime(df[col], format='%Y%m%d', errors='coerce')
```

```

42.         min_date = temp.min()
43.         df['day'] = (temp - min_date).dt.days.fillna(0).astype(int)
44.     else:
45.         df['day'] = df[col].astype(int)
46.     return df
47.
48.     def load_data(self):
49.         print("Loading data...")
50.         all_inf, all_mig = [], []
51.
52.         for city in self.cities:
53.             city_path = os.path.join(self.base_path, city)
54.             if not os.path.exists(city_path): continue
55.
56.             # Infection
57.             f_inf = os.path.join(city_path, 'infection.csv')
58.             if os.path.exists(f_inf):
59.                 df = pd.read_csv(f_inf, names=['date', 'region', 'cnt'], header
=None)
60.                 df = self.normalize_date(df)
61.                 daily = df.groupby('day')['cnt'].sum().reset_index()
62.                 daily['city'] = city
63.                 daily['cnt_smooth'] = daily['cnt'].rolling(window=3, min_period
s=1).mean()
64.                 all_inf.append(daily)
65.
66.             # Migration
67.             f_mig = os.path.join(city_path, 'migration.csv')
68.             if os.path.exists(f_mig):
69.                 df = pd.read_csv(f_mig, names=['date', 'f', 't', 'idx'], header
=None)
70.                 df = self.normalize_date(df)
71.                 daily_m = df.groupby('day')['idx'].sum().reset_index()
72.                 daily_m['city'] = city
73.                 daily_m['idx_smooth'] = daily_m['idx'].rolling(window=3, min_pe
riods=1).mean()
74.                 all_mig.append(daily_m)
75.
76.         df_inf = pd.concat(all_inf, ignore_index=True) if all_inf else pd.DataF
rame()
77.         df_mig = pd.concat(all_mig, ignore_index=True) if all_mig else pd.DataF
rame()
78.         return df_inf, df_mig
79.

```

```

80.
81. # =====
82. # 2. 可视化模块
83. # =====
84.
85. def plot_ridge_joyplot(df_inf):
86.     """图 1: 山脊图"""
87.     print("Plotting Ridge Plot...")
88.     if df_inf.empty: return
89.
90.     cities = df_inf['city'].unique()
91.     n_cities = len(cities)
92.     fig, axes = plt.subplots(n_cities, 1, figsize=(10, 8), sharex=True)
93.     if n_cities == 1: axes = [axes]
94.     fig.subplots_adjust(hspace=-0.4)
95.
96.     palette = sns.color_palette("coolwarm", n_cities)
97.     max_day = df_inf['day'].max()
98.
99.     for i, city in enumerate(cities):
100.         ax = axes[i]
101.         subset = df_inf[df_inf['city'] == city]
102.         ax.fill_between(subset['day'], 0, subset['cnt_smooth'], color=palette[i], alpha=0.85)
103.         ax.plot(subset['day'], subset['cnt_smooth'], color='white', lw=1.5)
104.         ax.text(0.01, 0.4, city, transform=ax.transAxes, fontweight='bold',
105.                fontsize=12, ha='left')
106.         ax.set_yticks([])
107.         ax.set_ylabel('')
108.         ax.spines['left'].set_visible(False)
109.         ax.spines['bottom'].set_visible(False)
110.         ax.patch.set_alpha(0)
111.     axes[-1].spines['bottom'].set_visible(True)
112.     axes[-1].set_xlabel('Relative Day', fontweight='bold')
113.     axes[-1].set_xlim(0, max_day)
114.     fig.suptitle('Epidemic Peaks Across Cities', y=0.92, fontsize=16, fontweight='bold')
115.     plt.show()
116.
117.
118. def plot_standardized_heatmap(df_inf):
119.     """图 3: 热力图"""

```



```

120.     print("Plotting Normalized Heatmap...")
121.     if df_inf.empty: return
122.
123.     pivot = df_inf.pivot_table(index='city', columns='day', values='cnt_smooth', aggfunc='sum').fillna(0)
124.     # Z-Score
125.     pivot_norm = pivot.apply(lambda x: (x - x.mean()) / (x.std() + 1e-5), axis=1)
126.
127.     plt.figure(figsize=(12, 5))
128.     sns.heatmap(pivot_norm, cmap='RdBu_r', center=0, vmin=-2, vmax=3,
129.                 linewidths=0.5, linecolor='white',
130.                 cbar_kws={'label': 'Z-Score'})
131.
132.     plt.title('Spatiotemporal Synchronization (Standardized)', fontsize=14, pad=15)
133.     plt.xlabel('Relative Day', fontweight='bold')
134.     plt.ylabel('City ID', fontweight='bold')
135.     plt.yticks(rotation=0)
136.     plt.tight_layout()
137.     plt.show()
138.
139.
140. def plot_bubble_matrix(df_inf, df_mig):
141.     """
142.     图 2 (新版): 时序气泡矩阵图 (Temporal Bubble Matrix)
143.
144.     维度编码:
145.     - X 轴: 时间 (Day)
146.     - Y 轴: 城市 (City)
147.     - 气泡大小: 感染人数 (Infection) -> 重点突出发病规模
148.     - 气泡颜色: 迁徙指数 (Migration) -> 展示流动性强弱
149.     """
150.     print("Plotting Bubble Matrix...")
151.     if df_inf.empty or df_mig.empty: return
152.
153.     # 1. 数据合并
154.     merged = pd.merge(df_inf, df_mig, on=['city', 'day'], how='inner')
155.     if merged.empty: return
156.
157.     # 2. 数据平滑 (让视觉效果更好)
158.     # 对每个城市分别计算平滑
159.     merged['cnt_smooth'] = merged.groupby('city')['cnt'].transform(lambda x:
        x.rolling(3, 1).mean())

```

```

160.     merged['idx_smooth'] = merged.groupby('city')['idx'].transform(lambda x:
    x.rolling(3, 1).mean())
161.
162.     # 3. 准备绘图
163.     fig, ax = plt.subplots(figsize=(13, 7))
164.
165.     cities = merged['city'].unique()
166.     # 将城市转为数字索引以便绘图: 0, 1, 2...
167.     city_map = {city: i for i, city in enumerate(cities)}
168.     merged['city_y'] = merged['city'].map(city_map)
169.
170.     # 4. 定义气泡大小 (Size Scaling)
171.     # 由于 City A 可能有 10 万, City E 只有 100, 线性映射会导致 City E 看不见
172.     # 这里使用 power scale (0.5 次方 即开根号) 或者 log scale 来压缩差距
173.     # 也可以简单地除以最大值 * 因子
174.     max_inf = merged['cnt_smooth'].max()
175.     # s = (val / max) * scale
176.     # 为了让小城市也能被看见, 加一个基础大小 base_size
177.     base_size = 30
178.     scale_factor = 800
179.     # 使用平方根缩放, 既能体现差异, 又不会让大球大得离谱
180.     merged['size'] = (np.sqrt(merged['cnt_smooth']) / np.sqrt(max_inf)) * scale_factor + base_size
181.
182.     # 5. 绘制散点 (Scatter)
183.     # c=idx_smooth (颜色映射迁徙), s=size (大小映射感染)
184.     scatter = ax.scatter(
185.         x=merged['day'],
186.         y=merged['city_y'],
187.         s=merged['size'],
188.         c=merged['idx_smooth'],
189.         cmap='YlGnBu', # 浅黄到深蓝, 蓝色代表高迁徙
190.         alpha=0.85,
191.         edgecolors='gray',
192.         linewidth=0.5
193.     )
194.
195.     # 6. 美化图表
196.     # 设置 Y 轴标签为城市名
197.     ax.set_yticks(range(len(cities)))
198.     ax.set_yticklabels(cities, fontweight='bold', fontsize=12)
199.
200.     # 设置 X 轴
201.     ax.set_xlabel('Time Evolution (Relative Day)', fontweight='bold', fontsi

```

```

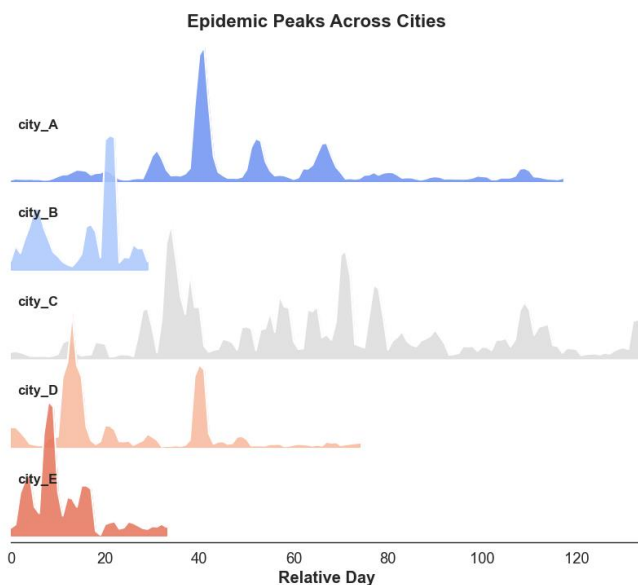
ze=12)
202.     ax.set_xlim(merged['day'].min() - 1, merged['day'].max() + 3) # 留点边
    距
203.
204.     # 添加网格线 (只显示 x 轴方向的虚线, 方便看时间对齐)
205.     ax.grid(True, axis='x', linestyle='--', alpha=0.5)
206.     # 去除多余边框
207.     sns.despine(left=True, bottom=False)
208.
209.     # --- 添加图例 (Legends) ---
210.
211.     # 7.1 颜色条 (Colorbar) -> 代表迁徙指数
212.     cbar = plt.colorbar(scatter, ax=ax, pad=0.02)
213.     cbar.set_label('Migration Index (Color Depth)', fontweight='bold')
214.
215.     # 7.2 大小图例 (Size Legend) -> 代表感染人数
216.     # 制作几个示例大小: 最大值的 10%, 50%, 100%
217.     # 逆运算还原真实数值
218.     legend_sizes = [0.1, 0.5, 1.0]
219.     legend_labels = [int(max_inf * r) for r in legend_sizes]
220.     # 计算对应的气泡显示大小
221.     legend_area_sizes = [(np.sqrt(val) / np.sqrt(max_inf)) * scale_factor +
        base_size for val in legend_labels]
222.
223.     # 手动创建图例句柄
224.     legend_elements = []
225.     for area, label in zip(legend_area_sizes, legend_labels):
226.         legend_elements.append(Line2D([0], [0], marker='o', color='w',
227.             markerfacecolor='gray', label=f'{label}
        '),
228.             markersize=np.sqrt(area), alpha=0.6))
        # markersize 大概是面积的开方
229.
230.     # 添加大小图例到上方
231.     legend1 = ax.legend(handles=legend_elements, title="New Infections (Bubble Size)",
        loc='upper center', bbox_to_anchor=(0.5, 1.15),
232.         ncol=3, frameon=False)
233.
234.
235.     ax.set_title('Spatio-Temporal Evolution: Migration vs. Infection', fontsize=14, y=1.15, fontweight='bold')
236.
237.     plt.tight_layout()
238.     plt.show()

```

```

239.
240.
241. # =====
242. # 3. 主程序入口
243. # =====
244. if __name__ == "__main__":
245.     BASE_DIR = r"C:\Users\21626\Downloads\train_data\train_data"
246.     TARGET_CITIES = ['city_A', 'city_B', 'city_C', 'city_D', 'city_E']
247.
248.     if os.path.exists(BASE_DIR):
249.         loader = MultiCityLoader(BASE_DIR, TARGET_CITIES)
250.         df_inf, df_mig = loader.load_data()
251.
252.         if not df_inf.empty:
253.             plot_ridge_joyplot(df_inf)
254.             plot_standardized_heatmap(df_inf)
255.             plot_bubble_matrix(df_inf, df_mig)
256.             print("Done.")
257.         else:
258.             print("No data loaded.")

```



`plot_ridge_joyplot`: 山脊图（Joyplot）—— 多城市疫情曲线形态对比
核心功能

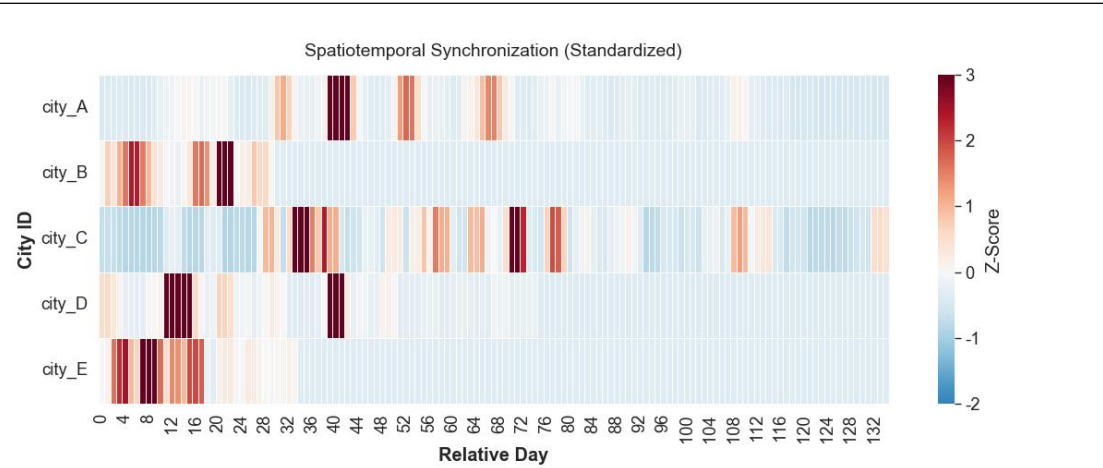
该图表聚焦多城市疫情时序曲线的形态对比，核心操作如下：

- 数据预处理：对每个城市的每日新增感染数做 3 天滚动平均（`cnt_smooth`），消除单日数据噪声，突出核心趋势；
- 布局设计：按城市数量垂直堆叠子图（ n 个城市 = n 个子图），共享 x 轴（相对天数），设置负间距（`hspace=-0.4`）形成“山脊重叠”的视觉效果；

• 视觉呈现：填充面积展示感染数规模，白色轮廓线强化曲线形态，每个子图左上角标注城市名；

意义

- 核心价值：形态化对比疫情传播特征无需复杂统计，一眼识别三个关键维度：
 - ① 峰值高度：填充面积越大 / 峰值越高，说明该城市疫情规模越大；
 - ② 峰值时间：曲线峰值对应的 x 轴天数，对比各城市疫情爆发的早晚（如 city_A 峰值在 Day10，city_B 在 Day20）；
 - ③ 曲线形态：识别“陡升陡降”（传播快、管控见效快）或“平缓波动”（传播慢、持续时间长）的城市；
- 学术价值：展示多城市疫情传播的异质性，为后续“城市特征归因分析”提供直观依据；
- 业务价值：
- 快速定位“高风险城市”（峰值高、上升斜率大），优先部署管控资源；
- 识别“慢扩散城市”的特征（如曲线平缓），提炼其防控经验；



plot_standardized_heatmap: 标准化热力图（Z-Score）—— 疫情时空同步性分析

核心功能

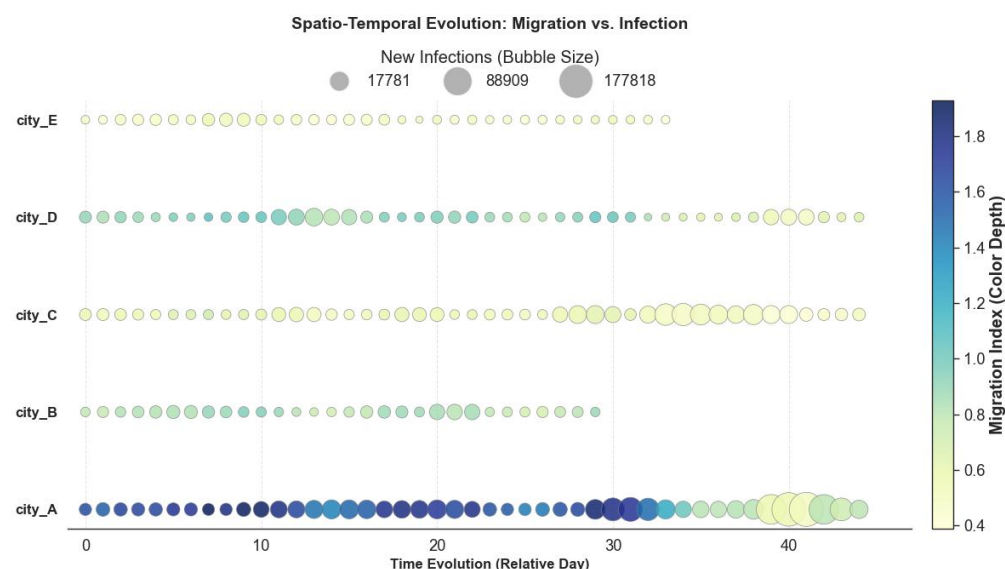
该图表解决“不同城市感染数量级差异”的问题，聚焦多城市疫情的时空同步性，核心操作如下：

- 数据重构：将“城市 × 天数 × 感染数”的长格式数据转为宽格式透视表（行 = 城市，列 = 天数，值 = 平滑感染数）；
- 标准化处理：按行（每个城市）计算 Z-Score $(x - \text{均值}) / \text{标准差}$ ，消除不同城市感染数绝对值差异（如 city_A 日感染数 1000，city_B 仅 100）；
- 可视化呈现：热力图展示标准化后的值，配色 RdBu_r（红色 = 高于自身均值，蓝色 = 低于自身均值），限定 vmin=-2、vmax=3 避免极端值干扰；
- 样式优化：添加白色网格线，Y 轴标签水平显示，提升可读性。

意义

- 核心价值：量化疫情的时空同步性
- ① 列方向（天数）：某一列大面积红色，说明多数城市在该天感染数高于自身均值（疫情“同步爆发”）；
- ② 行方向（城市）：某一行连续红色，说明该城市长期处于高感染状态（管控效果差）；

- ③ 局部模式：识别“同步高峰”（多城市同一列红色）或“异步波动”（各城市峰值列不同）；
- 学术价值：量化“时空同步性”是疫情传播研究的核心方向，该图为“区域间疫情传播的关联性”提供直观证据，符合顶刊“量化 + 可视化”的分析要求；
 - 业务价值：
 - 协同管控：识别“同步爆发期”，启动跨城市协同防控（如统一封控、核酸筛查）；
 - 异常排查：某城市 Z-Score 持续偏高（红色），需排查管控漏洞（如人员流动未限制）；
 - 公平评估：标准化后避免“大城市感染数高就被误判为风险高”，精准评估管控效果。



plot_bubble_matrix: 时序气泡矩阵图 —— 迁徙 - 感染的时空关联分析

核心功能

该图表是多维度时空关联可视化，同时编码“时间、城市、感染数、迁徙指数”四个维度，核心操作如下：

- 数据整合：合并感染 / 迁徙数据，按城市做滚动平滑，保证时间维度对齐；
- 多维度编码：

X 轴：相对天数（时间维度）；

Y 轴：城市（空间维度）；

- 气泡大小：感染数（平方根缩放 + 基础大小，避免小值气泡不可见）；
- 气泡颜色：迁徙指数（YlGnBu 配色，深蓝 = 高迁徙）；
- 图例设计：手动创建大小图例（对应感染数）+ 颜色条（对应迁徙指数），解决多维度编码的解读歧义；

意义

- 核心价值：直观验证“迁徙 - 感染”的时空关联

- ① 同一城市：观察气泡颜色（迁徙）变化是否先于大小（感染）变化（验证迁徙对感染的滞后效应，如迁徙指数升高后 7 天感染数上升）；
- ② 同一时间：对比不同城市气泡的大小 / 颜色，识别“高迁徙 - 高感染”的城市（重点管控对象）；
- ③ 整体模式：识别“迁徙高峰后感染高峰”的普遍规律，或偏离该规律的城市（如高迁徙但低感染，可能管控到位）；

•学术价值：将“时间、空间、两个核心指标”整合在一张图中，为“迁徙驱动疫情传播”提供直观的时空证据，符合顶刊“多维度整合分析”的要求； •业务价值： 精准管控：快速定位“高迁徙 + 高感染”的城市 - 时间节点，针对性强化管控（如封控、交通限制）； 政策评估：识别“高迁徙但低感染”的城市，提炼其管控经验（如迁徙管控 + 核酸筛查）； 趋势预判：若某城市气泡颜色加深（迁徙增加），可预判后续感染数可能上升，提前干预；
➤ 山脊图：单维度（感染数）时序对比，聚焦“形态异质性”； ➤ 标准化热力图：标准化后聚焦“时空同步性”； ➤ 气泡矩阵图：多维度整合，验证“迁徙 - 感染”的时空关联；三者从“描述”到“验证”，完整覆盖多城市疫情分析的核心逻辑。